

HAFIZ QURAN TUTOR — ONLINE QURAN LEARNING PLATFORM

Backend API Reference Guide

Complete REST API documentation for all 13 backend modules

Version 1.0 · Generated 2026-09-04

NestJS 12 · Prisma 7 · PostgreSQL

162 documented endpoints across 11 business domains

Table of Contents

Hafiz Quran Tutor — Backend API Reference	6
How to read this guide	6
Auth	7
POST /api/v1/auth/register	7
POST /api/v1/auth/login	7
POST /api/v1/auth/refresh	8
POST /api/v1/auth/logout	8
GET /api/v1/auth/me	8
People	9
Admins	9
POST /api/v1/people/admins	9
GET /api/v1/people/admins/me	9
Teachers	10
POST /api/v1/people/teachers	10
GET /api/v1/people/teachers/me	10
PATCH /api/v1/people/teachers/me	10
GET /api/v1/people/teachers	11
GET /api/v1/people/teachers/:id	11
GET /api/v1/people/teachers/admin/list	11
GET /api/v1/people/teachers/admin/:id	12
PATCH /api/v1/people/teachers/:id	12
Parents	12
POST /api/v1/people/parents/me	12
GET /api/v1/people/parents/me	13
PATCH /api/v1/people/parents/me	13
POST /api/v1/people/parents/me/children	13
GET /api/v1/people/parents/me/children	14
GET /api/v1/people/parents	14
GET /api/v1/people/parents/:id	14
PATCH /api/v1/people/parents/:id	15
Students	15
POST /api/v1/people/students/me	15
GET /api/v1/people/students/me	15
PATCH /api/v1/people/students/me	16
GET /api/v1/people/students	16
GET /api/v1/people/students/:id	16
PATCH /api/v1/people/students/:id	16
Courses	17
POST /api/v1/courses	17
GET /api/v1/courses	17
GET /api/v1/courses/:slug	18
GET /api/v1/courses/admin/list	18
GET /api/v1/courses/admin/:id	19
PATCH /api/v1/courses/:id	19
DELETE /api/v1/courses/:id	19

POST /api/v1/courses/:courseId/sections	19
PATCH /api/v1/courses/:courseId/sections/:sectionId	20
DELETE /api/v1/courses/:courseId/sections/:sectionId	20
POST /api/v1/courses/:courseId/faqs	20
PATCH /api/v1/courses/:courseId/faqs/:faqId	21
DELETE /api/v1/courses/:courseId/faqs/:faqId	21
POST /api/v1/courses/:courseId/teachers/:teacherId	21
DELETE /api/v1/courses/:courseId/teachers/:teacherId	21
Enrollment	22
POST /api/v1/enrollments	22
GET /api/v1/enrollments	23
GET /api/v1/enrollments/me	23
GET /api/v1/enrollments/me/:id	23
GET /api/v1/enrollments/children/:studentId	23
GET /api/v1/enrollments/:id	24
PATCH /api/v1/enrollments/:id	24
POST /api/v1/enrollments/:id/status	24
PATCH /api/v1/enrollments/:id/teacher	25
Scheduling	25
POST /api/v1/scheduling/availability/rules	25
GET /api/v1/scheduling/availability/rules/me	26
PATCH /api/v1/scheduling/availability/rules/:id	26
DELETE /api/v1/scheduling/availability/rules/:id	26
POST /api/v1/scheduling/availability/exceptions	27
GET /api/v1/scheduling/availability/exceptions/me	27
DELETE /api/v1/scheduling/availability/exceptions/:id	27
GET /api/v1/scheduling/availability/teachers/:teacherId	27
POST /api/v1/scheduling/schedules	28
GET /api/v1/scheduling/schedules	28
GET /api/v1/scheduling/schedules/:id	29
PATCH /api/v1/scheduling/schedules/:id	29
POST /api/v1/scheduling/schedules/:id/generate-sessions	29
POST /api/v1/scheduling/sessions	30
GET /api/v1/scheduling/sessions	30
GET /api/v1/scheduling/sessions/student/me	31
GET /api/v1/scheduling/sessions/teacher/me	31
GET /api/v1/scheduling/sessions/children/:studentId	31
GET /api/v1/scheduling/sessions/:id	31
PATCH /api/v1/scheduling/sessions/:id/cancel	32
PATCH /api/v1/scheduling/sessions/:id/reschedule	32
Learning	33
POST /api/v1/learning/sessions/:sessionId/attendance	33
PATCH /api/v1/learning/sessions/:sessionId/attendance	33
GET /api/v1/learning/sessions/:sessionId/attendance	34
POST /api/v1/learning/sessions/:sessionId/lesson	34
PATCH /api/v1/learning/sessions/:sessionId/lesson	34
GET /api/v1/learning/sessions/:sessionId/lesson	35
POST /api/v1/learning/homework	35

GET /api/v1/learning/homework/teacher/me	36
GET /api/v1/learning/homework/student/me	36
GET /api/v1/learning/homework/children/:studentId	36
GET /api/v1/learning/homework/:id	36
PATCH /api/v1/learning/homework/:id	37
POST /api/v1/learning/homework/:id/submissions	37
PATCH /api/v1/learning/homework/submissions/:submissionId/review	37
GET /api/v1/learning/progress/enrollment/:enrollmentId	38
GET /api/v1/learning/progress/enrollment/:enrollmentId/history	38
PATCH /api/v1/learning/progress/enrollment/:enrollmentId	39
CRM	39
Trial Requests	40
POST /api/v1/crm/trial-requests	40
GET /api/v1/crm/trial-requests	40
GET /api/v1/crm/trial-requests/:id	41
PATCH /api/v1/crm/trial-requests/:id	41
POST /api/v1/crm/trial-requests/:id/convert	41
POST /api/v1/crm/trial-requests/:id/sessions	42
GET /api/v1/crm/trial-requests/:id/sessions	43
Trial Sessions	43
PATCH /api/v1/crm/trial-sessions/:id	43
Contact Inquiries	44
POST /api/v1/crm/contact-inquiries	44
GET /api/v1/crm/contact-inquiries	44
GET /api/v1/crm/contact-inquiries/:id	44
PATCH /api/v1/crm/contact-inquiries/:id	45
Communication	45
POST /api/v1/messaging/conversations	45
GET /api/v1/messaging/conversations/me	46
GET /api/v1/messaging/conversations/:id/messages	46
POST /api/v1/messaging/conversations/:id/messages	46
PATCH /api/v1/messaging/conversations/:id/read	47
POST /api/v1/announcements	47
GET /api/v1/announcements/admin	48
GET /api/v1/announcements/me	48
PATCH /api/v1/announcements/:id	48
GET /api/v1/notifications/me	49
PATCH /api/v1/notifications/:id/read	49
PATCH /api/v1/notifications/read-all	49
Billing	50
POST /api/v1/billing/packages	50
GET /api/v1/billing/packages	51
GET /api/v1/billing/packages/admin	51
PATCH /api/v1/billing/packages/:id	51
POST /api/v1/billing/invoices	51
GET /api/v1/billing/invoices	52
GET /api/v1/billing/invoices/me	52
GET /api/v1/billing/invoices/children/:studentId	53

GET /api/v1/billing/invoices/:id	53
PATCH /api/v1/billing/invoices/:id	53
POST /api/v1/billing/invoices/:id/payments	54
GET /api/v1/billing/invoices/:id/payments	54
GET /api/v1/billing/payments	54
CMS	55
Testimonials	55
POST /api/v1/cms/testimonials	55
GET /api/v1/cms/testimonials	56
GET /api/v1/cms/testimonials/admin	56
PATCH /api/v1/cms/testimonials/:id	56
DELETE /api/v1/cms/testimonials/:id	56
FAQs	57
POST /api/v1/cms/faqs	57
GET /api/v1/cms/faqs	57
GET /api/v1/cms/faqs/admin	58
PATCH /api/v1/cms/faqs/:id	58
DELETE /api/v1/cms/faqs/:id	58
Website Content	59
GET /api/v1/cms/content/:key	59
GET /api/v1/cms/content/admin/list	59
GET /api/v1/cms/content/admin/:key	59
PUT /api/v1/cms/content/:key	60
Platform	60
POST /api/v1/media/upload	60
GET /api/v1/media/entity/:entityType/:entityId	61
GET /api/v1/media/:id	62
GET /api/v1/media/:id/file	62
DELETE /api/v1/media/:id	63
POST /api/v1/media/:id/attach	63
GET /api/v1/audit/logs	63
GET /api/v1/settings	64
GET /api/v1/settings/:key	64
PUT /api/v1/settings/:key	65

Hafiz Quran Tutor — Backend API Reference

Version: 1.0 **Base URL:** `{{baseUrl}}` (e.g. `http://localhost:3000` in local development) **API prefix:** all routes below are mounted under `/api/v1`, except the two health-check routes (`/health/live`, `/health/ready`), which are not versioned.

How to read this guide

Every route requires a valid JWT access token by default. Routes explicitly marked **Public** skip authentication entirely. Routes that require authentication but list no specific role are open to **any authenticated user**, regardless of role.

Roles are: `SUPER_ADMIN`, `ADMIN`, `TEACHER`, `PARENT`, `STUDENT`.

Authenticate by sending `Authorization: Bearer <accessToken>` on every request, where `accessToken` comes from `POST /api/v1/auth/login` or `POST /api/v1/auth/register`. Access tokens are short-lived; call `POST /api/v1/auth/refresh` with the paired refresh token to get a new one.

Every non-2xx response uses the same error envelope:

```
{
  "error": {
    "code": "BAD_REQUEST",
    "message": "Validation failed",
    "fieldErrors": [{ "field": "email", "message": "email must be an email" }],
    "requestId": "..."
  }
}
```

`fieldErrors` is present only for request-body validation failures (400s from the global `ValidationPipe`); `requestId` echoes the `x-request-id` header when the caller sent one.

Paginated list endpoints share one response shape:

```
{
  "data": [ /* array of items */ ],
  "meta": { "page": 1, "limit": 20, "total": 57, "totalPages": 3 }
}
```

Where an endpoint's ownership/authorization check fails for a resource that also might not exist, this backend consistently returns `404 Not Found` rather than `403 Forbidden` — this is deliberate: it avoids leaking to an unauthorized caller whether a given resource id exists at all. This convention is called out explicitly wherever it applies below.

Domains are documented in the order a resource moves through the platform's core lifecycle: **Visitor** → **Lead (CRM)** → **Trial** → **Student (Auth/People)** → **Course enrollment** → **Assigned Teacher** → **Scheduled Classes (Scheduling)** → **Learning** → **Progress** → **Payment (Billing)** → **Renewal**, bracketed by the cross-cutting Communication, CMS, and Platform domains.

Auth

The Auth domain issues and validates the credentials every other domain relies on. Access control uses short-lived signed JWTs (`accessToken`, default 15 minutes, `JWT_ACCESS_EXPIRES_IN`) paired with long-lived, database-backed opaque refresh tokens (default 30 days, `JWT_REFRESH_EXPIRES_IN`) in the format "`<sessionId>.<randomSecret>`". Refresh tokens are never JWTs — the session id looks up a `UserSession` row and the secret is bcrypt-compared against a stored hash, so sessions are individually revocable (logout, or automatically on suspected reuse) without a second signing key. Every refresh **rotates**: the presented session is revoked and a brand-new access/refresh pair is issued, so a refresh token can only be used once.

POST /api/v1/auth/register

Auth: Public

Description: Self-service account creation for the public-facing site. Restricted to `STUDENT` and `PARENT` roles — `TEACHER` accounts are always admin-provisioned (see People) and `ADMIN/SUPER_ADMIN` accounts are provisioned out of band.

Request body: | Field | Type | Required | Notes | |---|---|---|---| | `email` | string | no | *Valid email format.* | | `phone` | string | no | **At least one of `email`/`phone` is required (enforced in the service, not the DTO — omitting both returns a 400).* | | `password` | string | yes | *Minimum 8 characters.* | | `role` | string | yes | *Must be `STUDENT` or `PARENT`.* |

Response (201):

```
{ "id": "3f1c...-uuid", "email": "parent@example.com", "phone": null }
```

Errors: - `400 Bad Request` — neither `email` nor `phone` provided; or `role` is not `STUDENT/PARENT` (e.g. attempting to self-register as `TEACHER`). - `409 Conflict` — an account with that email or phone already exists.

POST /api/v1/auth/login

Auth: Public

Description: Authenticates with email-or-phone + password and issues a fresh access/refresh token pair. Also records `lastLoginAt` and, if `deviceId` is supplied, associates the session with that device.

Request body: | Field | Type | Required | Notes | |---|---|---|---| | `email` | string | conditional | *Required if `phone` is omitted. Valid email format.* | | `phone` | string | conditional | *Required if `email` is omitted.* | | `password` | string | yes | | | `deviceId` | string | no | *Client-supplied device identifier, stored on the session.* |

Response (200):

```
{
  "accessToken": "eyJhbGciOi...",
  "refreshToken": "9f2c1a3e-...-uuid.a1b2c3...",
  "expiresIn": "15m"
}
```

Errors: - **401 Unauthorized** — no matching account, wrong password, or the account has no password set. - **401 Unauthorized** — account **status** is not **ACTIVE** (e.g. suspended).

POST /api/v1/auth/refresh

Auth: Public (the refresh token itself is the credential)

Description: Exchanges a still-valid refresh token for a brand-new access/refresh pair, revoking the old session (rotation). Reusing an already-consumed or unknown refresh token is treated as a possible token-theft signal: the associated session (if any) is defensively revoked.

Request body: | Field | Type | Required | Notes | |---|---|---|---| | refreshToken | string | yes | The full "<sessionId>.<secret>" string from a previous login/refresh. |

Response (200): new token pair, same shape as login.

Errors: - **401 Unauthorized** — malformed token (missing the `.` separator), unknown/expired/already-revoked session, secret mismatch, or the owning account is no longer active.

POST /api/v1/auth/logout

Auth: Public (the refresh token itself is the credential)

Description: Revokes the session backing the given refresh token. Idempotent — logging out twice, or with an already-invalid token, still returns 204 (no error is raised for an unknown session id).

Request body: | Field | Type | Required | Notes | |---|---|---|---| | refreshToken | string | yes | The session to revoke. |

Response (204): no content.

Errors: none — always succeeds from the caller's perspective.

GET /api/v1/auth/me

Auth: Bearer token required (any authenticated user)

Description: Returns the calling account's own identity and role list, re-read from the database (not just decoded from the token) — reflects the account's current **status** and roles.

Response (200):

```
{
  "id": "3f1c...-uuid",
  "email": "parent@example.com",
  "phone": null,
  "status": "ACTIVE",
  "roles": ["PARENT"]
}
```

Errors: none specific — an invalid/expired access token is rejected by the global guard before this handler runs (401).

People

The People domain owns the four profile types layered on top of an Auth **User**: **ADMIN**, **TEACHER**, **PARENT**, and **STUDENT**. Admin and Teacher accounts are always provisioned by an existing admin (credentials + profile created together in one call); Parents and Students self-serve their own profile after registering through Auth. A Parent can additionally manage one or more "child" Student profiles that have no login credentials of their own — just a bare, credential-less **User** row tied to the parent via a relationship record.

Admins

POST /api/v1/people/admins

Auth: Bearer token required (SUPER_ADMIN only)

Description: Provisions a new plain **ADMIN** account (credentials + profile in one call). Deliberately restricted to SUPER_ADMIN so the admin roster can't be grown by an existing ADMIN — the API never creates another SUPER_ADMIN.

Request body: | Field | Type | Required | Notes | |---|---|---|---| | email | string | no | *Valid email format.* | | phone | string | no | **At least one of email/phone is required.* | | password | string | yes | *Minimum 8 characters.* | | firstName | string | yes | *Non-empty.* | | lastName | string | yes | *Non-empty.* | | jobTitle | string | no | |

Response (201):

```
{
  "id": "admin-profile-uuid",
  "userId": "3f1c...-uuid",
  "firstName": "Bilal",
  "lastName": "Hussain",
  "jobTitle": "Operations Admin",
  "createdAt": "2026-09-03T10:00:00.000Z",
  "updatedAt": "2026-09-03T10:00:00.000Z"
}
```

Errors: - **400 Bad Request** — neither email nor phone provided. - **409 Conflict** — an account with that email or phone already exists.

GET /api/v1/people/admins/me

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Returns the caller's own admin profile.

Response (200): same shape as the create response above.

Errors: - **404 Not Found** — the caller's account has the ADMIN/SUPER_ADMIN role but no **AdminProfile** row exists yet.

Teachers

POST /api/v1/people/teachers

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Admin-only. Provisions a login-capable **TEACHER** account and its public-facing profile in one call.

Request body: | Field | Type | Required | Notes | |---|---|---|---| | email | string | no | *Valid email.* | | phone | string | no | *At least one of **email/phone** required. | | password | string | yes | Minimum 8 characters. | | firstName | string | yes | Non-empty. | | lastName | string | yes | Non-empty. | | bio | string | yes | Non-empty, full biography. | | shortBio | string | no | Short teaser bio for listings. | | qualification | string | yes | Non-empty. | | experienceYears | number | no | Integer, ≥ 0 . | | teachingPhilosophy | string | no | | | countryCode | string | yes | ISO 3166-1 alpha-2 (e.g. **PK**). | | timezone | string | yes | Valid IANA timezone name (e.g. **Asia/Karachi**). |

Response (201):

```
{
  "id": "teacher-profile-uuid",
  "userId": "3f1c...-uuid",
  "firstName": "Ahmed",
  "lastName": "Khan",
  "bio": "Ten years teaching Quran recitation and Tajweed to students worldwide.",
  "shortBio": null,
  "qualification": "Ijazah in Quran recitation, Al-Azhar University",
  "experienceYears": 10,
  "teachingPhilosophy": null,
  "countryCode": "PK",
  "timezone": "Asia/Karachi",
  "status": "ACTIVE",
  "createdAt": "2026-09-03T10:00:00.000Z",
  "updatedAt": "2026-09-03T10:00:00.000Z"
}
```

Errors: - **400 Bad Request** — neither email nor phone; invalid **countryCode/timezone**. - **409 Conflict** — an account with that email or phone already exists.

GET /api/v1/people/teachers/me

Auth: Bearer token required (TEACHER only)

Description: Returns the caller's own teacher profile.

Response (200): same shape as the create response.

Errors: - **404 Not Found** — no teacher profile exists for this account yet.

PATCH /api/v1/people/teachers/me

Auth: Bearer token required (TEACHER only)

Description: Self-service profile edit. Cannot change `status` — that's admin-only (see `PATCH /api/v1/people/teachers/:id`).

Request body: all fields from the create DTO, all optional (partial update). | Field | Type | Required | Notes |
|---|---|---|---| | firstName | string | no | Non-empty if provided. | | lastName | string | no | Non-empty if provided. | | bio | string | no | Non-empty if provided. | | shortBio | string | no | | | qualification | string | no | Non-empty if provided. | | experienceYears | number | no | Integer ≥ 0. | | teachingPhilosophy | string | no | | | countryCode | string | no | ISO 3166-1 alpha-2. | | timezone | string | no | Valid IANA timezone. |

Response (200): updated profile, same shape as create.

Errors: - `404 Not Found` — no teacher profile exists for this account yet.

GET /api/v1/people/teachers

Auth: Public

Description: Public website "Teachers" page listing — visitors browse without logging in. Only `ACTIVE` teachers are included.

Query params: - `page` (number, optional, default 1) - `limit` (number, optional, default 20, max 100)

Response (200):

```
{
  "data": [ { "id": "teacher-profile-uuid", "firstName": "Ahmed", "lastName": "Khan", "status": "ACTIVE", "
  "meta": { "page": 1, "limit": 20, "total": 12, "totalPages": 1 }
}
```

Errors: none specific.

GET /api/v1/people/teachers/:id

Auth: Public

Description: Public teacher detail page. Only returns the profile if it is `ACTIVE` — inactive teachers 404 here even though they still exist (admins use the separate `admin/:id` route to see them).

Path params: - `id` (string, uuid) — teacher profile id

Response (200): same shape as the teacher profile object above.

Errors: - `404 Not Found` — no such teacher, or the teacher exists but is `INACTIVE`.

GET /api/v1/people/teachers/admin/list

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Admin listing of every teacher, including `INACTIVE` ones (distinct from the public `GET /people/teachers` route which hides inactive teachers).

Query params: - `page` (number, optional, default 1) - `limit` (number, optional, default 20, max 100)

Response (200): paginated list, same shape as the public listing but unfiltered by status.

Errors: none specific.

GET /api/v1/people/teachers/admin/:id

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Admin detail lookup by id — works regardless of status (ACTIVE or INACTIVE).

Path params: - `id` (string, uuid) — teacher profile id

Response (200): teacher profile object.

Errors: - 404 Not Found — no teacher with that id.

PATCH /api/v1/people/teachers/:id

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Admin edit of any teacher's profile — the only route that can also flip `status` between `ACTIVE/INACTIVE` (e.g. deactivating a teacher hides them from public listings without deleting the account).

Path params: - `id` (string, uuid) — teacher profile id

Request body: same optional fields as the self-service update, plus: | Field | Type | Required | Notes | |---|---|---|---| | status | string | no | `ACTIVE` or `INACTIVE`. |

Response (200): updated teacher profile.

Errors: - 404 Not Found — no teacher with that id.

Parents

POST /api/v1/people/parents/me

Auth: Bearer token required (PARENT only)

Description: Creates the caller's own parent profile after registering. One profile per account.

Request body: | Field | Type | Required | Notes | |---|---|---|---| | firstName | string | yes | Non-empty. | | lastName | string | no | | | countryCode | string | yes | ISO 3166-1 alpha-2. | | timezone | string | yes | Valid IANA timezone. |

Response (201):

```
{
  "id": "parent-profile-uuid",
  "userId": "3f1c...-uuid",
  "firstName": "Fatima",
  "lastName": "Ali",
  "countryCode": "AE",
  "timezone": "Asia/Dubai",
```

```
"createdAt": "2026-09-03T10:00:00.000Z",
"updatedAt": "2026-09-03T10:00:00.000Z"
}
```

Errors: - **409 Conflict** — a parent profile already exists for this account.

GET /api/v1/people/parents/me

Auth: Bearer token required (PARENT only)

Description: Returns the caller's own parent profile.

Response (200): same shape as create response.

Errors: - **404 Not Found** — no parent profile exists for this account yet.

PATCH /api/v1/people/parents/me

Auth: Bearer token required (PARENT only)

Description: Self-service profile edit.

Request body: all fields from create, optional. | Field | Type | Required | Notes | |---|---|---|---| | firstName | string | no | Non-empty if provided. | | lastName | string | no | | countryCode | string | no | ISO 3166-1 alpha-2. | | timezone | string | no | Valid IANA timezone. |

Response (200): updated profile.

Errors: - **404 Not Found** — no parent profile exists yet.

POST /api/v1/people/parents/me/children

Auth: Bearer token required (PARENT only)

Description: Adds a child the parent manages directly. Creates a bare, credential-less **User** row (email/phone/password all null) purely to satisfy the student profile's required **userId**, plus the student profile itself and the parent↔student relationship link — all in one transaction.

Request body: | Field | Type | Required | Notes | |---|---|---|---| | firstName | string | yes | Non-empty. | | lastName | string | no | | dateOfBirth | string | no | ISO date, e.g. "2015-04-12". | | gender | string | no | **MALE** or **FEMALE**. | | countryCode | string | yes | ISO 3166-1 alpha-2. | | timezone | string | yes | Valid IANA timezone. | | relationshipType | string | yes | **MOTHER**, **FATHER**, **GUARDIAN**, or **OTHER**. | | isPrimary | boolean | no | Default **true**. | | canViewProgress | boolean | no | Default **true**. | | canViewPayments | boolean | no | Default **true**. | | canJoinClass | boolean | no | Default **true**. |

Response (201):

```
{
  "relationshipId": "rel-uuid",
  "relationshipType": "FATHER",
  "isPrimary": true,
```

```

"canViewProgress": true,
"canViewPayments": true,
"canJoinClass": true,
"student": {
  "id": "student-profile-uuid",
  "userId": "child-user-uuid",
  "firstName": "Yusuf",
  "lastName": "Ali",
  "dateOfBirth": "2015-06-01",
  "gender": "MALE",
  "countryCode": "AE",
  "timezone": "Asia/Dubai",
  "status": "ACTIVE"
}
}

```

Errors: - **400 Bad Request** — invalid `timezone/countryCode/relationshipType/gender`, or the **STUDENT** role isn't seeded in the database yet. - **404 Not Found** — no parent profile exists for the caller yet (must create one via **POST** `/people/parents/me` first).

GET `/api/v1/people/parents/me/children`

Auth: Bearer token required (PARENT only)

Description: Lists every child the caller manages, oldest-added first.

Response (200): array of child relationship objects, same shape as the add-child response (not paginated).

```

[ { "relationshipId": "rel-uuid", "student": { "firstName": "Yusuf", "...": "...", "...": "..." } } ]

```

Errors: - **404 Not Found** — no parent profile exists for the caller yet.

GET `/api/v1/people/parents`

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Admin listing of every parent profile.

Query params: - **page** (number, optional, default 1) - **limit** (number, optional, default 20, max 100)

Response (200): paginated list of parent profiles (same shape as the create response's item).

Errors: none specific.

GET `/api/v1/people/parents/:id`

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Admin lookup of a single parent profile by id.

Path params: - **id** (string, uuid) — parent profile id

Response (200): parent profile object.

Errors: - 404 Not Found — no parent with that id.

PATCH /api/v1/people/parents/:id

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Admin edit of any parent's profile. Uses the same DTO as self-service update — parent profiles have no **status** field to gate, unlike teachers/students.

Path params: - **id** (string, uuid) — parent profile id

Request body: same optional fields as PATCH /people/parents/me.

Response (200): updated parent profile.

Errors: - 404 Not Found — no parent with that id.

Students

POST /api/v1/people/students/me

Auth: Bearer token required (STUDENT only)

Description: Creates the caller's own student profile (used by self-registered students, as opposed to children added by a parent). One profile per account.

Request body: | Field | Type | Required | Notes | |---|---|---|---| | firstName | string | yes | Non-empty. | | lastName | string | no | | | dateOfBirth | string | no | ISO date. Age is derived from this on demand, never stored separately. | | gender | string | no | MALE or FEMALE. | | countryCode | string | yes | ISO 3166-1 alpha-2. | | timezone | string | yes | Valid IANA timezone. |

Response (201):

```
{
  "id": "student-profile-uuid",
  "userId": "3f1c...-uuid",
  "firstName": "Yusuf",
  "lastName": "Ali",
  "dateOfBirth": "2015-06-01",
  "gender": "MALE",
  "countryCode": "AE",
  "timezone": "Asia/Dubai",
  "status": "ACTIVE",
  "createdAt": "2026-09-03T10:00:00.000Z",
  "updatedAt": "2026-09-03T10:00:00.000Z"
}
```

Errors: - 409 Conflict — a student profile already exists for this account.

GET /api/v1/people/students/me

Auth: Bearer token required (STUDENT only)

Description: Returns the caller's own student profile.

Response (200): same shape as create response.

Errors: - **404 Not Found** — no student profile exists for this account yet.

PATCH /api/v1/people/students/me

Auth: Bearer token required (STUDENT only)

Description: Self-service profile edit — same fields as creation, all optional.

Request body: | Field | Type | Required | Notes | |---|---|---|---| | firstName | string | no | Non-empty if provided. | | lastName | string | no | | dateOfBirth | string | no | ISO date. | | gender | string | no | **MALE** or **FEMALE**. | | countryCode | string | no | ISO 3166-1 alpha-2. | | timezone | string | no | Valid IANA timezone. |

Response (200): updated profile.

Errors: - **404 Not Found** — no student profile exists yet.

GET /api/v1/people/students

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Admin listing of every student profile (both self-registered students and parent-managed children).

Query params: - **page** (number, optional, default 1) - **limit** (number, optional, default 20, max 100)

Response (200): paginated list of student profiles.

Errors: none specific.

GET /api/v1/people/students/:id

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Admin lookup of a single student profile by id.

Path params: - **id** (string, uuid) — student profile id

Response (200): student profile object.

Errors: - **404 Not Found** — no student with that id.

PATCH /api/v1/people/students/:id

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Admin edit of any student's profile — the only route that can flip **status** between **ACTIVE/INACTIVE** for a student.

Path params: - **id** (string, uuid) — student profile id

Request body: same optional fields as self-service update, plus: | Field | Type | Required | Notes | |---|---|---|---| | status | string | no | **ACTIVE** or **INACTIVE**. |

Response (200): updated student profile.

Errors: - **404 Not Found** — no student with that id.

Courses

The Courses domain models the public course catalog: top-level **Course** records (each with a unique **slug** for its public URL) that move through **DRAFT** → **PUBLISHED** → **HIDDEN** states, plus nested curriculum sections, FAQs, and teacher assignments. Only **PUBLISHED** courses (and their sections/FAQs/teachers) are visible on the public, unauthenticated routes; admins can see and edit courses in any state, including soft-deleted ones being excluded entirely.

POST /api/v1/courses

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Creates a new course. Always starts in **DRAFT** status — invisible on public routes until explicitly published via the update endpoint.

Request body: | Field | Type | Required | Notes | |---|---|---|---| | slug | string | yes | Lowercase, alphanumeric, hyphen-separated (e.g. "**quran-reading**"); must be unique. | | name | string | yes | Non-empty. | | shortDescription | string | yes | Non-empty. | | description | string | yes | Non-empty, full description. | | suitableFor | string | no | | | ageGroup | string | no | | | teachingMethod | string | no | | | classFormat | string | no | | | sortOrder | number | no | Integer ≥ 0; defaults to 0. |

Response (201):

```
{
  "id": "course-uuid",
  "slug": "postman-course-123",
  "name": "Postman Test Course",
  "shortDescription": "A course created from the Postman collection.",
  "suitableFor": "Anyone testing the API.",
  "ageGroup": "All ages",
  "status": "DRAFT",
  "sortOrder": 0,
  "createdAt": "2026-09-03T10:00:00.000Z",
  "updatedAt": "2026-09-03T10:00:00.000Z"
}
```

Errors: - **400 Bad Request** — **slug** doesn't match the lowercase-hyphen pattern. - **409 Conflict** — a course with that slug already exists.

GET /api/v1/courses

Auth: Public

Description: Public website course catalog listing. Only **PUBLISHED**, non-deleted courses, ordered by **sortOrder**.

Query params: - **page** (number, optional, default 1) - **limit** (number, optional, default 20, max 100)

Response (200):

```
{
  "data": [ { "id": "course-uuid", "slug": "hifz-ul-quran", "name": "Hifz-ul-Quran", "status": "PUBLISHED",
  "meta": { "page": 1, "limit": 20, "total": 6, "totalPages": 1 }
}
```

Errors: none specific.

GET /api/v1/courses/:slug

Auth: Public

Description: Public course detail page — includes curriculum sections, FAQs, and assigned teachers. Only resolves **PUBLISHED**, non-deleted courses.

Path params: - **slug** (string) — the course's unique slug

Response (200):

```
{
  "id": "course-uuid",
  "slug": "hifz-ul-quran",
  "name": "Hifz-ul-Quran",
  "shortDescription": "Memorize the Quran with a certified teacher.",
  "description": "Full description...",
  "suitableFor": "All ages",
  "ageGroup": "All ages",
  "teachingMethod": "One-on-one live video",
  "classFormat": "Online",
  "status": "PUBLISHED",
  "sortOrder": 4,
  "sections": [ { "id": "section-uuid", "title": "Week 1: Introduction", "description": "Getting started.",
  "faqs": [ { "id": "faq-uuid", "question": "Is this course beginner-friendly?", "answer": "Yes.", "sortOrd
  "teachers": [ { "id": "teacher-profile-uuid", "firstName": "Ahmed", "lastName": "Khan", "shortBio": null
  "createdAt": "2026-01-01T00:00:00.000Z",
  "updatedAt": "2026-01-01T00:00:00.000Z"
}
```

Errors: - **404 Not Found** — no such course, the course is **DRAFT/HIDDEN**, or it has been soft-deleted (all three cases look identical to the caller).

GET /api/v1/courses/admin/list

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Admin catalog listing — includes **DRAFT** and **HIDDEN** courses (soft-deleted ones still excluded), which the public **GET /courses** route hides.

Query params: - **page** (number, optional, default 1) - **limit** (number, optional, default 20, max 100)

Response (200): paginated list, same item shape as the public listing but unfiltered by status.

Errors: none specific.

GET /api/v1/courses/admin/:id

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Admin detail lookup by id (not slug) — works for any non-deleted course regardless of status, including sections/FAQs/teachers.

Path params: - `id` (string, uuid) — course id

Response (200): same shape as the public detail response.

Errors: - `404 Not Found` — no such course, or it has been soft-deleted.

PATCH /api/v1/courses/:id

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Edits a course. This single endpoint also drives publish/hide transitions via the `status` field — there's no separate publish/hide route.

Path params: - `id` (string, uuid) — course id

Request body: all fields from create, optional, plus: | Field | Type | Required | Notes | |---|---|---|---| | slug | string | no | Lowercase-hyphen pattern; must stay unique. | | name | string | no | | | shortDescription | string | no | | | description | string | no | | | suitableFor | string | no | | | ageGroup | string | no | | | teachingMethod | string | no | | | classFormat | string | no | | | sortOrder | number | no | | | status | string | no | `DRAFT`, `PUBLISHED`, or `HIDDEN`. |

Response (200): updated course (list-item shape, not the full detail with sections/faqs/teachers).

Errors: - `404 Not Found` — no such course, or it's soft-deleted. - `409 Conflict` — the new `slug` is already used by a different course.

DELETE /api/v1/courses/:id

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Soft-deletes a course (sets `deletedAt`). After this, even the admin detail route (`admin/:id`) returns 404 for it.

Path params: - `id` (string, uuid) — course id

Response (204): no content.

Errors: - `404 Not Found` — no such course, or already deleted.

POST /api/v1/courses/:courseId/sections

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Adds a curriculum section to a course.

Path params: - `courseId` (string, uuid) — parent course id

Request body: | Field | Type | Required | Notes | |---|---|---|---| | title | string | yes | Non-empty. | | description | string | yes | Non-empty. | | sortOrder | number | no | Integer ≥ 0 ; defaults to 0. |

Response (201):

```
{ "id": "section-uuid", "title": "Week 1: Introduction", "description": "Getting started.", "sortOrder": 1
```

Errors: - `404 Not Found` — no such course, or it's soft-deleted.

PATCH /api/v1/courses/:courseId/sections/:sectionId

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Edits a curriculum section.

Path params: - `courseId` (string, uuid) — parent course id - `sectionId` (string, uuid) — section id

Request body: all create fields, optional. | Field | Type | Required | Notes | |---|---|---|---| | title | string | no | Non-empty if provided. | | description | string | no | Non-empty if provided. | | sortOrder | number | no | |

Response (200): updated section.

Errors: - `404 Not Found` — no section with that id under that course.

DELETE /api/v1/courses/:courseId/sections/:sectionId

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Removes a curriculum section.

Path params: - `courseId` (string, uuid) — parent course id - `sectionId` (string, uuid) — section id

Response (204): no content.

Errors: - `404 Not Found` — no section with that id under that course.

POST /api/v1/courses/:courseId/faqs

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Adds an FAQ entry to a course.

Path params: - `courseId` (string, uuid) — parent course id

Request body: | Field | Type | Required | Notes | |---|---|---|---| | question | string | yes | Non-empty. | | answer | string | yes | Non-empty. | | sortOrder | number | no | Integer ≥ 0 ; defaults to 0. |

Response (201):

```
{ "id": "faq-uuid", "question": "Is this course beginner-friendly?", "answer": "Yes, no prior experience n
```

Errors: - 404 Not Found — no such course, or it's soft-deleted.

PATCH /api/v1/courses/:courseId/faqs/:faqId

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Edits an FAQ entry.

Path params: - `courseId` (string, uuid) — parent course id - `faqId` (string, uuid) — FAQ id

Request body: all create fields, optional. | Field | Type | Required | Notes | |---|---|---|---| | question | string | no | Non-empty if provided. | | answer | string | no | Non-empty if provided. | | sortOrder | number | no | |

Response (200): updated FAQ.

Errors: - 404 Not Found — no FAQ with that id under that course.

DELETE /api/v1/courses/:courseId/faqs/:faqId

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Removes an FAQ entry.

Path params: - `courseId` (string, uuid) — parent course id - `faqId` (string, uuid) — FAQ id

Response (204): no content.

Errors: - 404 Not Found — no FAQ with that id under that course.

POST /api/v1/courses/:courseId/teachers/:teacherId

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Assigns a teacher to a course (many-to-many). No request body.

Path params: - `courseId` (string, uuid) — course id - `teacherId` (string, uuid) — teacher profile id

Response (204): no content.

Errors: - 404 Not Found — no such course, or no such teacher profile. - 409 Conflict — the teacher is already assigned to this course.

DELETE /api/v1/courses/:courseId/teachers/:teacherId

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Unassigns a teacher from a course.

Path params: - `courseId` (string, uuid) — course id - `teacherId` (string, uuid) — teacher profile id

Response (204): no content.

Errors: - 404 Not Found — the teacher is not currently assigned to this course.

Enrollment

The Enrollment domain models a student's relationship to a course: linking a **StudentProfile** to a **Course** (and usually a **TeacherProfile**), tracking its lifecycle status (**TRIAL** → **ACTIVE** → **PAUSED/COMPLETED/CANCELLED**), and recording teacher (re)assignments. It's the anchor resource that Scheduling (class schedules/sessions) and Learning (attendance/lessons/homework/progress) both hang off of via **enrollmentId**. Enrollment management itself is admin-only; students and parents get narrow, read-only, ownership-scoped views.

POST /api/v1/enrollments

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Creates a new enrollment linking a student to a course, optionally with an initial teacher. If a teacher is supplied, an initial **EnrollmentTeacherAssignment** history row is created alongside it in the same transaction.

Request body: | Field | Type | Required | Notes | |---|---|---|---| | studentId | string (uuid) | yes | Must reference an existing **StudentProfile**. | | courseId | string (uuid) | yes | Must reference an existing, non-deleted **Course**. | | teacherId | string (uuid) | no | Must reference a **TeacherProfile** with **status: ACTIVE**. | | status | string | no | One of **TRIAL**, **PENDING**. An enrollment can only ever be *created* in these two statuses — every other status is reached via the status-transition endpoint. Defaults to **PENDING**. | | studentTimezone | string | yes | IANA timezone string (e.g. **Asia/Karachi**), validated by the custom **IsIanaTimezone** validator. | | notes | string | no | Free-text admin notes. |

Response (201): the created enrollment, with student/course/teacher name summaries embedded.

```
{
  "id": "enr-uuid",
  "studentId": "student-uuid",
  "courseId": "course-uuid",
  "teacherId": "teacher-uuid",
  "status": "PENDING",
  "startedAt": null,
  "endedAt": null,
  "studentTimezone": "Asia/Karachi",
  "notes": null,
  "createdAt": "2026-09-03T10:00:00.000Z",
  "updatedAt": "2026-09-03T10:00:00.000Z",
  "student": { "firstName": "Ali", "lastName": "Khan" },
  "course": { "slug": "hifz-intensive", "name": "Hifz Intensive" },
  "teacher": { "firstName": "Sana", "lastName": "Malik" }
}
```

Errors: - 404 Not Found — **studentId** doesn't reference an existing student, or **courseId** doesn't reference an existing (non-deleted) course. - 404 Not Found — **teacherId** given but doesn't reference an existing teacher. - 400 Bad Request — **teacherId** given but that teacher's **status** isn't **ACTIVE**.

GET /api/v1/enrollments

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Lists all enrollments across the platform, with optional filters. Intended for admin back-office views.

Query params: - `page` (number, optional, default 1) - `limit` (number, optional, default 20, max 100) - `studentId` (string, uuid, optional) - `courseId` (string, uuid, optional) - `teacherId` (string, uuid, optional) - `status` (string, optional) — one of `TRIAL`, `PENDING`, `ACTIVE`, `PAUSED`, `COMPLETED`, `CANCELLED`

Response (200): paginated list, newest first.

```
{
  "data": [ { "id": "enr-uuid", "status": "ACTIVE", "...": "..." } ],
  "meta": { "page": 1, "limit": 20, "total": 3, "totalPages": 1 }
}
```

Errors: none specific — an unmatched filter combination simply returns an empty page.

GET /api/v1/enrollments/me

Auth: Bearer token required (STUDENT only)

Description: SRS "Student > View enrolled courses" — lists the caller's own enrollments. Resolved from the caller's own `StudentProfile`, not a path parameter, so a student can never pass someone else's id.

Query params: - `page` (number, optional, default 1) - `limit` (number, optional, default 20, max 100)

Response (200): paginated list of the caller's own enrollments, same shape as `GET /enrollments`.

Errors: - `404 Not Found` — the caller's account has no `StudentProfile` yet.

GET /api/v1/enrollments/me/:id

Auth: Bearer token required (STUDENT only)

Description: Fetches one of the caller's own enrollments by id.

Path params: - `id` (string, uuid) — the enrollment id

Response (200): single enrollment, same shape as the create response.

Errors: - `404 Not Found` — the caller has no `StudentProfile`, or the enrollment doesn't exist, or it exists but doesn't belong to the caller (indistinguishable from "doesn't exist", by design).

GET /api/v1/enrollments/children/:studentId

Auth: Bearer token required (PARENT only)

Description: SRS "Parent > View child's courses" — lists a specific child's enrollments. Access is relationship-checked via `ParentStudentRelationship`, not just role-checked: the caller must have a `ParentProfile` linked to the given `studentId`.

Path params: - `studentId` (string, uuid) — the child's student profile id

Query params: - `page` (number, optional, default 1) - `limit` (number, optional, default 20, max 100)

Response (200): paginated list of the child's enrollments, same shape as `GET /enrollments`.

Errors: - `404 Not Found` — the caller has no `ParentProfile`, or no `ParentStudentRelationship` links them to `studentId` (used to avoid leaking whether the student id exists at all).

GET /api/v1/enrollments/:id

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Fetches a single enrollment by id, admin view.

Path params: - `id` (string, uuid) — the enrollment id

Response (200): single enrollment, same shape as the create response.

Errors: - `404 Not Found` — no enrollment with that id.

PATCH /api/v1/enrollments/:id

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Edits plain fields on an enrollment (timezone, notes). Status changes are deliberately excluded from this endpoint and must go through `POST /:id/status` so every status change is validated against the allowed state machine.

Path params: - `id` (string, uuid) — the enrollment id

Request body: | Field | Type | Required | Notes | |---|---|---|---| | studentTimezone | string | no | IANA timezone string. | | notes | string | no | Replaces the notes field. |

Response (200): the updated enrollment.

Errors: - `404 Not Found` — no enrollment with that id.

POST /api/v1/enrollments/:id/status

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Transitions an enrollment's status through its lifecycle state machine. Returns `200`, not the POST default `201`, because this mutates an existing resource rather than creating one. Setting the target to `ACTIVE` stamps `startedAt` (only if not already set); setting it to `COMPLETED` or `CANCELLED` stamps `endedAt`. An optional `reason` is appended to the enrollment's `notes` field.

Allowed transitions: `TRIAL → ACTIVE|CANCELLED`, `PENDING → ACTIVE|CANCELLED`, `ACTIVE → PAUSED|COMPLETED|CANCELLED`, `PAUSED → ACTIVE|CANCELLED`. `COMPLETED` and `CANCELLED` are terminal — nothing transitions out of them.

Path params: - `id` (string, uuid) — the enrollment id

Request body: | Field | Type | Required | Notes | |---|---|---|---| | status | string | yes | One of **TRIAL**, **PENDING**, **ACTIVE**, **PAUSED**, **COMPLETED**, **CANCELLED**. Must be a valid next state per the transition table above. | | reason | string | no | Appended as a new line to the enrollment's **notes**. |

Response (200): the updated enrollment.

Errors: - **404 Not Found** — no enrollment with that id. - **409 Conflict** — the requested **status** isn't a valid transition from the enrollment's current status; the error message names the valid next states.

PATCH /api/v1/enrollments/:id/teacher

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Reassigns the teacher on an enrollment. Ends the current **EnrollmentTeacherAssignment** row (sets **endedAt**) and creates a new one recording the reason, all in one transaction.

Path params: - **id** (string, uuid) — the enrollment id

Request body: | Field | Type | Required | Notes | |---|---|---|---| | teacherId | string (uuid) | yes | Must reference a **TeacherProfile** with **status: ACTIVE**. | | reason | string | no | Recorded on the new **EnrollmentTeacherAssignment** row. |

Response (200): the updated enrollment with the new teacher.

Errors: - **404 Not Found** — no enrollment with that id, or **teacherId** doesn't reference an existing teacher. - **400 Bad Request** — the enrollment's status is **COMPLETED** or **CANCELLED** (can't reassign a teacher on a finished enrollment). - **400 Bad Request** — **teacherId** matches the teacher already assigned. - **400 Bad Request** — **teacherId** references a teacher whose **status** isn't **ACTIVE**.

Scheduling

The Scheduling domain has three layers: **availability** (a teacher's recurring weekly free/busy rules plus one-off exceptions), **class schedules** (a recurring weekly slot generated from an enrollment + teacher availability), and **class sessions** (the concrete, dated occurrences derived from a schedule, or created one-off). Sessions are what attendance, lesson records, and homework in the Learning domain actually attach to.

POST /api/v1/scheduling/availability/rules

Auth: Bearer token required (TEACHER only)

Description: Adds a recurring weekly availability rule for the calling teacher (e.g. "Mondays 14:00–16:00, Asia/Karachi"). Always scoped to the caller's own **TeacherProfile** — there's no admin-on-behalf-of variant of this write.

Request body: | Field | Type | Required | Notes | |---|---|---|---| | dayOfWeek | number | yes | **0–6**, **0** = Sunday, matching JS **Date**/Luxon weekday conventions. | | startTime | string | yes | "**HH:mm**", 24-hour, e.g. "**14:00**". Must be before **endTime**. | | endTime | string | yes | "**HH:mm**", 24-hour. | | timezone | string | yes | IANA timezone string. |

Response (201):

```
{
  "id": "rule-uuid",
  "dayOfWeek": 1,
  "startTime": "14:00",
  "endTime": "16:00",
  "timezone": "Asia/Karachi",
  "isActive": true
}
```

Errors: - **404 Not Found** — the caller's account has no **TeacherProfile** yet. - **400 Bad Request** — **startTime** is not strictly before **endTime**.

GET /api/v1/scheduling/availability/rules/me

Auth: Bearer token required (TEACHER only)

Description: Lists all of the caller's own recurring availability rules, ordered by day of week then start time.

Response (200): array of rules, same shape as the create response.

Errors: - **404 Not Found** — the caller's account has no **TeacherProfile** yet.

PATCH /api/v1/scheduling/availability/rules/:id

Auth: Bearer token required (TEACHER only)

Description: Toggles a rule active/inactive. Only **isActive** is editable here — the day/time/timezone are fixed once created (delete and recreate to change them).

Path params: - **id** (string, uuid) — the rule id

Request body: | Field | Type | Required | Notes | |---|---|---|---| | **isActive** | boolean | no | |

Response (200): the updated rule.

Errors: - **404 Not Found** — the caller has no **TeacherProfile**, or the rule doesn't exist, or it belongs to a different teacher (ownership-scoped lookup — indistinguishable from "doesn't exist").

DELETE /api/v1/scheduling/availability/rules/:id

Auth: Bearer token required (TEACHER only)

Description: Permanently deletes an availability rule.

Path params: - **id** (string, uuid) — the rule id

Response (204): no body.

Errors: - **404 Not Found** — the caller has no **TeacherProfile**, or the rule doesn't exist or isn't owned by the caller.

POST /api/v1/scheduling/availability/exceptions

Auth: Bearer token required (TEACHER only)

Description: Adds a one-off exception to the caller's normal weekly availability — either blocking out time (UNAVAILABLE, e.g. a holiday) or opening up extra time (EXTRA_AVAILABLE) for a specific date/time window.

Request body: | Field | Type | Required | Notes | |---|---|---|---| | startsAt | string (ISO 8601 datetime) | yes | Must be before endsAt. | | endsAt | string (ISO 8601 datetime) | yes | | | type | string | yes | UNAVAILABLE or EXTRA_AVAILABLE. | | reason | string | no | |

Response (201):

```
{
  "id": "exc-uuid",
  "startsAt": "2026-12-25T00:00:00.000Z",
  "endsAt": "2026-12-26T00:00:00.000Z",
  "type": "UNAVAILABLE",
  "reason": "Holiday"
}
```

Errors: - 404 Not Found — the caller's account has no TeacherProfile yet. - 400 Bad Request — startsAt is not strictly before endsAt.

GET /api/v1/scheduling/availability/exceptions/me

Auth: Bearer token required (TEACHER only)

Description: Lists the caller's own exceptions, ordered by startsAt ascending.

Response (200): array of exceptions, same shape as the create response.

Errors: - 404 Not Found — the caller's account has no TeacherProfile yet.

DELETE /api/v1/scheduling/availability/exceptions/:id

Auth: Bearer token required (TEACHER only)

Description: Permanently deletes an availability exception.

Path params: - id (string, uuid) — the exception id

Response (204): no body.

Errors: - 404 Not Found — the caller has no TeacherProfile, or the exception doesn't exist or isn't owned by the caller.

GET /api/v1/scheduling/availability/teachers/:teacherId

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Admin combined view of a teacher's rules and exceptions together — used when manually scheduling classes for that teacher.

Path params: - `teacherId` (string, uuid) — the teacher profile id

Response (200):

```
{
  "rules": [ { "id": "rule-uuid", "dayOfWeek": 1, "startTime": "14:00", "endTime": "16:00", "timezone": "As
  "exceptions": [ { "id": "exc-uuid", "startsAt": "2026-12-25T00:00:00.000Z", "endsAt": "2026-12-26T00:00:00.000Z" } ]
}
```

Errors: - `404 Not Found` — no `TeacherProfile` with that id.

POST /api/v1/scheduling/schedules

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Creates a recurring weekly class schedule for an enrollment. `studentId`, `teacherId`, and `courseId` are copied from the enrollment automatically — the caller only supplies the enrollment id and the recurrence details.

Request body: | Field | Type | Required | Notes | |---|---|---|---| | enrollmentId | string (uuid) | yes | Enrollment must already have a `teacherId` assigned, and must not be `COMPLETED/CANCELLED`. | | dayOfWeek | number | yes | `0-6`, `0` = Sunday. | | localStartTime | string | yes | `"HH:mm"`, 24-hour. | | durationMinutes | number | yes | `15-240`. | | timezone | string | yes | IANA timezone string. | | effectiveFrom | string (ISO date) | yes | First date the schedule applies from. | | effectiveUntil | string (ISO date) | no | Last date it applies; open-ended if omitted. |

Response (201): the created schedule, with student/teacher/course summaries embedded.

```
{
  "id": "sched-uuid",
  "enrollmentId": "enr-uuid",
  "studentId": "student-uuid",
  "teacherId": "teacher-uuid",
  "courseId": "course-uuid",
  "timezone": "Asia/Karachi",
  "dayOfWeek": 1,
  "localStartTime": "14:00",
  "durationMinutes": 60,
  "effectiveFrom": "2026-09-01",
  "effectiveUntil": null,
  "status": "ACTIVE",
  "student": { "firstName": "Ali", "lastName": "Khan" },
  "teacher": { "firstName": "Sana", "lastName": "Malik" },
  "course": { "slug": "hifz-intensive", "name": "Hifz Intensive" }
}
```

Errors: - `404 Not Found` — no enrollment with that id. - `400 Bad Request` — the enrollment has no `teacherId` assigned yet. - `400 Bad Request` — the enrollment's status is `COMPLETED` or `CANCELLED`.

GET /api/v1/scheduling/schedules

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Lists all class schedules across the platform, newest first.

Query params: - `page` (number, optional, default 1) - `limit` (number, optional, default 20, max 100)

Response (200): paginated list, same shape as the create response.

Errors: none specific.

GET /api/v1/scheduling/schedules/:id

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Fetches a single class schedule by id.

Path params: - `id` (string, uuid) — the schedule id

Response (200): single schedule, same shape as the create response.

Errors: - `404 Not Found` — no schedule with that id.

PATCH /api/v1/scheduling/schedules/:id

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Edits a schedule's recurrence details or status. Does not retroactively touch already-generated sessions.

Path params: - `id` (string, uuid) — the schedule id

Request body: | Field | Type | Required | Notes | |---|---|---|---| | dayOfWeek | number | no | 0–6. | | localStartTime | string | no | "HH:mm", 24-hour. | | durationMinutes | number | no | 15–240. | | effectiveUntil | string (ISO date) | no | | | status | string | no | One of `ACTIVE`, `PAUSED`, `ENDED`. |

Response (200): the updated schedule.

Errors: - `404 Not Found` — no schedule with that id.

POST /api/v1/scheduling/schedules/:id/generate-sessions

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Resolves the schedule's recurring local weekday/time into concrete UTC `ClassSession` rows for every matching date in `[from, to]`, intersected with the schedule's `effectiveFrom/effectiveUntil` window. Idempotent — re-running over an overlapping range skips dates that already have a session for this schedule (only newly-created sessions are returned, not pre-existing ones).

Path params: - `id` (string, uuid) — the schedule id

Request body: | Field | Type | Required | Notes | |---|---|---|---| | from | string (ISO date) | yes | Start of the generation range (inclusive), interpreted in the schedule's timezone. | | to | string (ISO date) | yes | End of the generation range (inclusive). Must not be before `from`. |

Response (201): array of newly-created sessions (empty array if none matched or all already existed).

```
[
  {
    "id": "sess-uuid",
    "classScheduleId": "sched-uuid",
    "enrollmentId": "enr-uuid",
    "studentId": "student-uuid",
    "teacherId": "teacher-uuid",
    "courseId": "course-uuid",
    "scheduledStartAt": "2026-09-07T09:00:00.000Z",
    "scheduledEndAt": "2026-09-07T10:00:00.000Z",
    "status": "SCHEDULED",
    "meetingProvider": "MANUAL",
    "student": { "firstName": "Ali", "lastName": "Khan" },
    "teacher": { "firstName": "Sana", "lastName": "Malik" },
    "course": { "slug": "hifz-intensive", "name": "Hifz Intensive" }
  }
]
```

Errors: - **404 Not Found** — no schedule with that id. - **400 Bad Request** — the schedule's status isn't **ACTIVE**. - **400 Bad Request** — **from/to** don't parse as valid dates in the schedule's timezone, or **from** is after **to**.

POST /api/v1/scheduling/sessions

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Creates a single one-off class session directly on an enrollment (not derived from a recurring schedule) — e.g. for a makeup class. **studentId/teacherId/courseId** are copied from the enrollment.

Request body: | Field | Type | Required | Notes | |---|---|---|---| enrollmentId | string (uuid) | yes | Enrollment must already have a **teacherId** assigned. | | scheduledStartAt | string (ISO 8601 datetime) | yes | Must be before **scheduledEndAt**. | | scheduledEndAt | string (ISO 8601 datetime) | yes | |

Response (201): the created session, same shape as one entry in the **generate-sessions** response (with **classScheduleId: null**).

Errors: - **404 Not Found** — no enrollment with that id. - **400 Bad Request** — the enrollment has no **teacherId** assigned yet. - **400 Bad Request** — **scheduledStartAt** is not strictly before **scheduledEndAt**.

GET /api/v1/scheduling/sessions

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Lists all class sessions across the platform, with optional filters, most recently scheduled first.

Query params: - **page** (number, optional, default 1) - **limit** (number, optional, default 20, max 100) - **studentId** (string, uuid, optional) - **teacherId** (string, uuid, optional) - **enrollmentId** (string, uuid, optional) - **status** (string, optional) — one of **SCHEDULED**, **COMPLETED**, **CANCELLED**, **RESCHEDULED**, **NO_SHOW**

Response (200): paginated list, same session shape as above.

Errors: none specific.

GET /api/v1/scheduling/sessions/student/me

Auth: Bearer token required (STUDENT only)

Description: Lists the caller's own class sessions, resolved from the caller's own `StudentProfile`.

Query params: - `page` (number, optional, default 1) - `limit` (number, optional, default 20, max 100)

Response (200): paginated list of the caller's own sessions.

Errors: - `404 Not Found` — the caller's account has no `StudentProfile` yet.

GET /api/v1/scheduling/sessions/teacher/me

Auth: Bearer token required (TEACHER only)

Description: Lists the caller's own assigned class sessions, resolved from the caller's own `TeacherProfile`.

Query params: - `page` (number, optional, default 1) - `limit` (number, optional, default 20, max 100)

Response (200): paginated list of the caller's own sessions.

Errors: - `404 Not Found` — the caller's account has no `TeacherProfile` yet.

GET /api/v1/scheduling/sessions/children/:studentId

Auth: Bearer token required (PARENT only)

Description: Lists a specific child's class sessions. Relationship-checked via `ParentStudentRelationship`, same pattern as the Enrollment domain's equivalent endpoint.

Path params: - `studentId` (string, uuid) — the child's student profile id

Query params: - `page` (number, optional, default 1) - `limit` (number, optional, default 20, max 100)

Response (200): paginated list of the child's sessions.

Errors: - `404 Not Found` — the caller has no `ParentProfile`, or no relationship links them to `studentId`.

GET /api/v1/scheduling/sessions/:id

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Fetches a single class session by id, admin view.

Path params: - `id` (string, uuid) — the session id

Response (200): single session, same shape as above.

Errors: - `404 Not Found` — no session with that id.

PATCH /api/v1/scheduling/sessions/:id/cancel

Auth: Bearer token required (ADMIN, SUPER_ADMIN, TEACHER)

Description: Cancels a scheduled session. A teacher may only cancel sessions they're assigned to teach — enforced by `ClassSessionService.assertCanManageSession`, which throws **404 Not Found** (not **403 Forbidden**) for another teacher's session, so a teacher can't use this endpoint to probe for the existence of sessions that aren't theirs. Admins can cancel any session.

Path params: - `id` (string, uuid) — the session id

Request body: | Field | Type | Required | Notes | |---|---|---|---| | reason | string | no | Recorded as `cancellationReason`. |

Response (200): the updated session, with `status`: "CANCELLED", `cancelledAt`, and `cancelledBy` set.

Errors: - **404 Not Found** — no session with that id, or the caller is a teacher who isn't assigned to it. - **400 Bad Request** — the session's current `status` isn't **SCHEDULED** (can't cancel an already-completed/cancelled/rescheduled/no-show session).

PATCH /api/v1/scheduling/sessions/:id/reschedule

Auth: Bearer token required (ADMIN, SUPER_ADMIN, TEACHER)

Description: Reschedules a session to a new start time. Implemented as create-new/mark-old rather than an in-place update: the original row is marked **RESCHEDULED** (never deleted or silently rewritten) and a brand-new **SCHEDULED** session is created carrying `rescheduledFromId` back to the original and `originalStartAt` forward from it (or from the original's own `scheduledStartAt` if this is the first reschedule in the chain). Duration is preserved from the original session. Same ownership rule as cancel: a teacher may only reschedule their own assigned sessions.

Path params: - `id` (string, uuid) — the session id being rescheduled

Request body: | Field | Type | Required | Notes | |---|---|---|---| | newStartAt | string (ISO 8601 datetime) | yes | The new scheduled start; end time is derived by preserving the original duration. | | reason | string | no | Recorded as `cancellationReason` on the *old* (now **RESCHEDULED**) row. |

Response (200): the newly-created session (not the old one), same shape as above, with `rescheduledFromId` pointing at the original.

Errors: - **404 Not Found** — no session with that id, or the caller is a teacher who isn't assigned to it. - **400 Bad Request** — the session's current `status` isn't **SCHEDULED**.

Learning

The Learning domain captures what actually happened in and around a class session: attendance, a teacher's lesson record, homework assignment/submission/review, and longitudinal student progress (current Surah/Ayah, tajweed/hifz scores) tied to an enrollment. Every read here is gated by `LearningAccessService.assertCanAccess` (or the near-identical inline check in `ProgressService`): admin, the assigned teacher, the owning student, or a linked parent — never role membership alone. Every teacher-only write is gated by `LearningAccessService.assertIsAssignedTeacher`, which resolves the caller's own `TeacherProfile` and compares it against the resource's `teacherId`. Both throw `404 Not Found` rather than `403 Forbidden` on failure, so an unauthorized caller can't tell "not yours" apart from "doesn't exist".

POST /api/v1/learning/sessions/:sessionId/attendance

Auth: Bearer token required (TEACHER only)

Description: Marks attendance for a class session. Only the session's assigned teacher may mark it (or an admin). One attendance record per session — use the `PATCH` endpoint to correct an existing one.

Path params: - `sessionId` (string, uuid) — the class session id

Request body: | Field | Type | Required | Notes | |---|---|---|---| | status | string | yes | One of `PRESENT`, `ABSENT`, `LATE`, `EXCUSED`. | | notes | string | no | |

Response (201):

```
{
  "id": "att-uuid",
  "classSessionId": "sess-uuid",
  "studentId": "student-uuid",
  "status": "PRESENT",
  "markedByTeacherId": "teacher-uuid",
  "markedAt": "2026-09-07T10:05:00.000Z",
  "notes": null
}
```

Errors: - `404 Not Found` — no session with that id, or the caller isn't the session's assigned teacher. - `409 Conflict` — attendance has already been marked for this session (use `PATCH` instead).

PATCH /api/v1/learning/sessions/:sessionId/attendance

Auth: Bearer token required (TEACHER only)

Description: Corrects an existing attendance record for a session. Same ownership rule as marking.

Path params: - `sessionId` (string, uuid) — the class session id

Request body: | Field | Type | Required | Notes | |---|---|---|---| | status | string | yes | One of `PRESENT`, `ABSENT`, `LATE`, `EXCUSED`. | | notes | string | no | |

Response (200): the updated attendance record.

Errors: - **404 Not Found** — no session with that id, the caller isn't the session's assigned teacher, or no attendance record exists yet for this session.

GET /api/v1/learning/sessions/:sessionId/attendance

Auth: Bearer token required (any of ADMIN, SUPER_ADMIN, TEACHER, STUDENT, PARENT — role list is intentionally wide open; `LearningAccessService.assertCanAccess` does the real gating on top)

Description: Fetches the attendance record for a session. Visible to admins, the session's assigned teacher, the enrolled student, or a linked parent.

Path params: - `sessionId` (string, uuid) — the class session id

Response (200): the attendance record, same shape as the mark response.

Errors: - **404 Not Found** — no session with that id, the caller isn't authorized to view it, or no attendance record has been recorded yet.

POST /api/v1/learning/sessions/:sessionId/lesson

Auth: Bearer token required (TEACHER only)

Description: Creates the lesson record for a session (what was taught, teacher notes, an optional 0–10 performance rating). One lesson record per session.

Path params: - `sessionId` (string, uuid) — the class session id

Request body: | Field | Type | Required | Notes | |---|---|---|---| | title | string | yes | Non-empty. | | content | string | yes | Non-empty. | | teacherNotes | string | no | | performanceRating | number | no | 0–10. | | generalRemarks | string | no | |

Response (201):

```
{
  "id": "lesson-uuid",
  "classSessionId": "sess-uuid",
  "studentId": "student-uuid",
  "teacherId": "teacher-uuid",
  "enrollmentId": "enr-uuid",
  "title": "Surah Al-Baqarah 1-10",
  "content": "Recited and corrected tajweed on...",
  "teacherNotes": null,
  "performanceRating": 8,
  "generalRemarks": null,
  "createdAt": "2026-09-07T10:10:00.000Z",
  "updatedAt": "2026-09-07T10:10:00.000Z"
}
```

Errors: - **404 Not Found** — no session with that id, or the caller isn't the session's assigned teacher. - **409 Conflict** — a lesson record already exists for this session (use `PATCH` instead).

PATCH /api/v1/learning/sessions/:sessionId/lesson

Auth: Bearer token required (TEACHER only)

Description: Updates the existing lesson record for a session. Same ownership rule as creation.

Path params: - `sessionId` (string, uuid) — the class session id

Request body: same fields as create (all still validated the same way when present).

Response (200): the updated lesson record.

Errors: - `404 Not Found` — no session with that id, the caller isn't the session's assigned teacher, or no lesson record exists yet for this session.

GET /api/v1/learning/sessions/:sessionId/lesson

Auth: Bearer token required (any of ADMIN, SUPER_ADMIN, TEACHER, STUDENT, PARENT; `LearningAccessService.assertCanAccess` gates the actual access)

Description: Fetches the lesson record for a session. Same visibility rule as attendance.

Path params: - `sessionId` (string, uuid) — the class session id

Response (200): the lesson record, same shape as the create response.

Errors: - `404 Not Found` — no session with that id, the caller isn't authorized to view it, or no lesson record has been recorded yet.

POST /api/v1/learning/homework

Auth: Bearer token required (TEACHER only)

Description: Assigns a new homework item to a student, tied to their enrollment and optionally to a specific class session. Only the enrollment's assigned teacher may create it.

Request body: | Field | Type | Required | Notes | |---|---|---|---| | enrollmentId | string (uuid) | yes | Enrollment must already have a `teacherId` assigned. | | classSessionId | string (uuid) | no | Optionally ties the homework to a specific session. | | title | string | yes | Non-empty. | | description | string | yes | Non-empty. | | dueAt | string (ISO 8601 datetime) | no | |

Response (201): the created homework, with an initially empty `submissions` array.

```
{
  "id": "hw-uuid",
  "enrollmentId": "enr-uuid",
  "studentId": "student-uuid",
  "teacherId": "teacher-uuid",
  "classSessionId": null,
  "title": "Memorize Surah Al-Fatiha",
  "description": "Recite from memory next session",
  "dueAt": "2026-09-14T00:00:00.000Z",
  "status": "ASSIGNED",
  "createdAt": "2026-09-07T10:15:00.000Z",
  "updatedAt": "2026-09-07T10:15:00.000Z",
  "submissions": []
}
```

Errors: - **404 Not Found** — no enrollment with that id, or the caller isn't the enrollment's assigned teacher. - **400 Bad Request** — the enrollment has no **teacherId** assigned yet.

GET /api/v1/learning/homework/teacher/me

Auth: Bearer token required (TEACHER only)

Description: Lists all homework the caller has assigned, resolved from the caller's own **TeacherProfile**.

Query params: - **page** (number, optional, default 1) - **limit** (number, optional, default 20, max 100)

Response (200): paginated list, newest first, each including its **submissions** array.

Errors: - **404 Not Found** — the caller's account has no **TeacherProfile** yet.

GET /api/v1/learning/homework/student/me

Auth: Bearer token required (STUDENT only)

Description: Lists the caller's own assigned homework, resolved from the caller's own **StudentProfile**.

Query params: - **page** (number, optional, default 1) - **limit** (number, optional, default 20, max 100)

Response (200): paginated list of the caller's own homework.

Errors: - **404 Not Found** — the caller's account has no **StudentProfile** yet.

GET /api/v1/learning/homework/children/:studentId

Auth: Bearer token required (PARENT only)

Description: Lists a specific child's homework. Relationship-checked via **ParentStudentRelationship**.

Path params: - **studentId** (string, uuid) — the child's student profile id

Query params: - **page** (number, optional, default 1) - **limit** (number, optional, default 20, max 100)

Response (200): paginated list of the child's homework.

Errors: - **404 Not Found** — the caller has no **ParentProfile**, or no relationship links them to **studentId**.

GET /api/v1/learning/homework/:id

Auth: Bearer token required (any of ADMIN, SUPER_ADMIN, TEACHER, STUDENT, PARENT; **LearningAccessService.assertCanAccess** gates the actual access)

Description: Fetches a single homework item by id, including its submissions.

Path params: - **id** (string, uuid) — the homework id

Response (200): single homework item, same shape as the create response.

Errors: - **404 Not Found** — no homework with that id, or the caller isn't authorized to view it.

PATCH /api/v1/learning/homework/:id

Auth: Bearer token required (TEACHER only)

Description: Edits a homework item's fields or status. Only the assigning teacher (or an admin, via the same `assertIsAssignedTeacher` check) may edit it.

Path params: - `id` (string, uuid) — the homework id

Request body: | Field | Type | Required | Notes | |---|---|---|---| | title | string | no | | | description | string | no | | | dueAt | string (ISO 8601 datetime) | no | | | status | string | no | One of `ASSIGNED`, `IN_PROGRESS`, `SUBMITTED`, `COMPLETED`, `OVERDUE`. |

Response (200): the updated homework item.

Errors: - `404 Not Found` — no homework with that id, or the caller isn't the assigned teacher.

POST /api/v1/learning/homework/:id/submissions

Auth: Bearer token required (STUDENT only)

Description: Submits the caller's work for a homework item. Only the homework's own student may submit (checked directly against the caller's `StudentProfile`, not via `LearningAccessService`, since this is a narrower "you're the owner" check rather than the general access rule). Also flips the parent homework's `status` to `SUBMITTED` in the same transaction.

Path params: - `id` (string, uuid) — the homework id

Request body: | Field | Type | Required | Notes | |---|---|---|---| | content | string | no | The submission text/link. |

Response (201):

```
{
  "id": "sub-uuid",
  "homeworkId": "hw-uuid",
  "studentId": "student-uuid",
  "submittedAt": "2026-09-13T18:00:00.000Z",
  "content": "Done, ready to recite.",
  "status": "PENDING",
  "teacherFeedback": null,
  "reviewedAt": null
}
```

Errors: - `404 Not Found` — no homework with that id, or the caller isn't its assigned student. - `400 Bad Request` — the homework's `status` is already `COMPLETED`.

PATCH /api/v1/learning/homework/submissions/:submissionId/review

Auth: Bearer token required (TEACHER only)

Description: Reviews a student's homework submission. Approving (`APPROVED`) marks the parent homework `COMPLETED`; requesting changes (`NEEDS_REVISION`) marks it `IN_PROGRESS` so the student can resubmit. Only the homework's assigned teacher may review.

Path params: - `submissionId` (string, uuid) — the submission id

Request body: | Field | Type | Required | Notes | |---|---|---|---| | status | string | yes | `APPROVED` or `NEEDS_REVISION`. | | teacherFeedback | string | no | |

Response (200): the updated submission, with `reviewedAt` stamped.

Errors: - `404 Not Found` — no submission with that id, or the caller isn't the parent homework's assigned teacher.

GET /api/v1/learning/progress/enrollment/:enrollmentId

Auth: Bearer token required (any of ADMIN, SUPER_ADMIN, TEACHER, STUDENT, PARENT — role list is wide; the actual gate is the inline `assertCanView` check below)

Description: Fetches the current progress snapshot for an enrollment (current lesson/Surah/Ayah, tajweed/hifz/performance scores). Visibility follows the same admin/assigned-teacher/owning-student pattern as elsewhere, but a linked parent additionally needs the `canViewProgress` flag set on their specific `ParentStudentRelationship` row — the one piece of data in this domain gated by a relationship-level permission flag rather than the relationship's mere existence.

Path params: - `enrollmentId` (string, uuid) — the enrollment id

Response (200):

```
{
  "id": "prog-uuid",
  "enrollmentId": "enr-uuid",
  "currentLesson": "Juz 5",
  "currentSurah": 4,
  "currentAyah": 23,
  "tajweedProgress": 72.5,
  "hifzProgress": 40,
  "performance": 85,
  "remarks": "Improving steadily",
  "updatedByTeacherId": "teacher-uuid",
  "createdAt": "2026-08-01T09:00:00.000Z",
  "updatedAt": "2026-09-07T10:20:00.000Z"
}
```

Errors: - `404 Not Found` — no enrollment with that id, the caller isn't authorized to view it (including a linked parent whose relationship has `canViewProgress: false`), or no progress has been recorded for this enrollment yet.

GET /api/v1/learning/progress/enrollment/:enrollmentId/history

Auth: Bearer token required (any of ADMIN, SUPER_ADMIN, TEACHER, STUDENT, PARENT; same `assertCanView` gate, including the `canViewProgress` relationship flag for parents)

Description: Fetches the full history of progress snapshots for an enrollment — one entry per `PATCH` update ever made, newest first. Unlike the current-progress endpoint, an empty history is not an error (returns []).

Path params: - `enrollmentId` (string, uuid) — the enrollment id

Response (200): array of history entries (no `id` linking back to the current progress row beyond `enrollmentId` — each entry is an independent append-only snapshot).

```
[
  {
    "id": "hist-uuid",
    "currentLesson": "Juz 5",
    "currentSurah": 4,
    "currentAyah": 23,
    "tajweedProgress": 72.5,
    "hifzProgress": 40,
    "performance": 85,
    "remarks": "Improving steadily",
    "changedByTeacherId": "teacher-uuid",
    "createdAt": "2026-09-07T10:20:00.000Z"
  }
]
```

Errors: - 404 Not Found — no enrollment with that id, or the caller isn't authorized to view it.

PATCH /api/v1/learning/progress/enrollment/:enrollmentId

Auth: Bearer token required (TEACHER only)

Description: Upserts the current progress snapshot for an enrollment and appends a new entry to [ProgressHistory](#) in the same transaction — so history is never lost, even though the "current" row is a single upserted record. Only the enrollment's assigned teacher may update it.

Path params: - [enrollmentId](#) (string, uuid) — the enrollment id

Request body: | Field | Type | Required | Notes | |---|---|---|---| | currentLesson | string | no | | | currentSurah | number | no | 1–114. | | currentAyah | number | no | >= 1. | | tajweedProgress | number | no | 0–100. | | hifzProgress | number | no | 0–100. | | performance | number | no | 0–100. | | remarks | string | no | |

Response (200): the updated (or newly-created) progress snapshot, same shape as [GET .../progress](#).

Errors: - 404 Not Found — no enrollment with that id, the enrollment has no [teacherId](#) assigned yet, or the caller isn't the assigned teacher.

CRM

The CRM domain implements the "Visitor → Lead → Trial → Student" pipeline that is central to the platform (SRS Section 55): an anonymous visitor submits a "Book Free Trial" form ([TrialRequest](#)) or a general "Contact Us" message ([ContactInquiry](#)); staff triage, assign, and schedule a [TrialSession](#) for the lead; and once the trial goes well, staff **convert** the trial request into a real [StudentProfile](#) and a TRIAL-status [Enrollment](#) in one atomic step. All three sub-resources are staff-only to read/manage (ADMIN, SUPER_ADMIN) — only the initial lead-capture submissions are public.

Trial Requests

POST /api/v1/crm/trial-requests

Auth: Public

Description: The public "Book Free Trial" form and the primary lead-generation entry point for the whole platform (SRS Section 8). Creates a **TrialRequest** in **NEW** status, tagged with **source: "website"**. The referenced course must exist and not be soft-deleted.

Request body: | Field | Type | Required | Notes | |---|---|---| | studentName | string | yes | Non-empty. | | studentAge | number | yes | Integer, 1–120. | | guardianName | string | no | | | countryCode | string | yes | ISO 3166-1 alpha-2 (e.g. **PK**). | | whatsapp | string | yes | Minimum 5 characters. | | email | string | no | | | phone | string | no | | | courseId | string (uuid) | yes | Must reference an existing, non-deleted course. | | preferredDays | string[] | yes | At least 1 entry (e.g. **["MON", "WED"]**). | | preferredTime | string | no | **HH:mm**, 24-hour, e.g. **"17:30"**. | | timezone | string | yes | Valid IANA timezone, e.g. **Asia/Karachi**. | | message | string | no | | | specialRequirements | string | no | |

Response (201):

```
{
  "id": "tr-uuid",
  "studentName": "Ahmed",
  "studentAge": 9,
  "guardianName": "Bilal",
  "countryCode": "PK",
  "whatsapp": "+923001234567",
  "email": null,
  "phone": null,
  "courseId": "course-uuid",
  "preferredDays": ["MON", "WED"],
  "preferredTime": "17:30",
  "timezone": "Asia/Karachi",
  "message": null,
  "specialRequirements": null,
  "status": "NEW",
  "assignedAdminId": null,
  "convertedUserId": null,
  "convertedEnrollmentId": null,
  "source": "website",
  "createdAt": "2026-09-04T10:00:00.000Z",
  "updatedAt": "2026-09-04T10:00:00.000Z",
  "course": { "slug": "hifz-ul-quran", "name": "Hifz-ul-Quran" }
}
```

Errors: - **404 Not Found** — **courseId** does not reference an existing, non-deleted course. - **400 Bad Request** — **preferredDays** is empty, **countryCode/timezone** fail validation, etc.

GET /api/v1/crm/trial-requests

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Lists trial requests (the staff triage queue), newest first, with optional filters.

Query params: - `page` (number, optional, default 1) - `limit` (number, optional, default 20, max 100) - `status` (string, optional) — one of `NEW`, `CONTACTED`, `TRIAL_SCHEDULED`, `TRIAL_COMPLETED`, `ENROLLED`, `REJECTED`, `CLOSED` - `courseId` (string, uuid, optional)

Response (200): paginated list, same item shape as the create response.

```
{
  "data": [ { "id": "tr-uuid", "status": "NEW", "course": { "slug": "hifz-ul-quran", "name": "Hifz-ul-Quran" } },
  "meta": { "page": 1, "limit": 20, "total": 4, "totalPages": 1 }
}
```

Errors: none specific beyond generic auth/role failures.

GET /api/v1/crm/trial-requests/:id

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Fetches a single trial request by id.

Path params: - `id` (string, uuid) — the trial request's id

Response (200): same shape as the create response.

Errors: - `404 Not Found` — no trial request with that id.

PATCH /api/v1/crm/trial-requests/:id

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Updates a trial request's triage status and/or assigns it to a staff member. This is the general "move it along the pipeline" endpoint (e.g. `NEW` → `CONTACTED`) — the more specific `.../sessions` and `.../convert` endpoints below handle scheduling and conversion.

Path params: - `id` (string, uuid)

Request body: | Field | Type | Required | Notes | |---|---|---|---| | status | string | no | One of `NEW`, `CONTACTED`, `TRIAL_SCHEDULED`, `TRIAL_COMPLETED`, `ENROLLED`, `REJECTED`, `CLOSED`. | | assignedAdminId | string (uuid) | no | |

Response (200): same shape as the create response, with updated fields.

Errors: - `404 Not Found` — no trial request with that id. - `400 Bad Request` — `status` is not one of the allowed values.

POST /api/v1/crm/trial-requests/:id/convert

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: The final, most important step of the pipeline: converts a trial request into a real student and a live `Enrollment` (status `TRIAL`) in a single database transaction, then marks the trial request `ENROLLED`. There are **two conversion paths**, chosen by whether the request body includes `studentProfileId`:

- **New student (no `studentProfileId`):** creates a brand-new `User` (role `STUDENT`, status `ACTIVE`, no login credentials of its own — the same "managed profile" pattern used when a parent adds a child) and a `StudentProfile` seeded from the trial request's `studentName`, `countryCode`, and `timezone`. The new profile's id becomes the enrollment's `studentId`.
- **Existing student (`studentProfileId` provided):** looks up that `StudentProfile` (404 if not found) and reuses its `userId`/id directly — no new user is created. This is for a lead that already has (or turns out to have) an account, e.g. a sibling of an existing student.

Either way, an `Enrollment` is created with `status: "TRIAL"`, `courseId` copied from the trial request, `studentTimezone` copied from the trial request's `timezone`, and a `notes` field referencing the source trial request. The trial request is then updated to `status: "ENROLLED"`, `assignedAdminId` set to the calling admin, and `convertedUserId`/`convertedEnrollmentId` populated.

Path params: - `id` (string, uuid) — the trial request to convert

Request body: | Field | Type | Required | Notes | |---|---|---|---| | `studentProfileId` | string (uuid) | no | Omit to create a new managed student; provide to link to an existing `StudentProfile`. |

Response (200): the trial request, now converted.

```
{
  "id": "tr-uuid",
  "status": "ENROLLED",
  "assignedAdminId": "admin-user-uuid",
  "convertedUserId": "new-or-existing-user-uuid",
  "convertedEnrollmentId": "enrollment-uuid",
  "course": { "slug": "hifz-ul-quran", "name": "Hifz-ul-Quran" },
  "...": "... (remaining fields unchanged from create response)"
}
```

Errors: - `404 Not Found` — no trial request with that id. - `404 Not Found` — `studentProfileId` was provided but no such `StudentProfile` exists. - `400 Bad Request` — the trial request's `status` is already `ENROLLED` ("This trial request has already been converted") — conversion is not idempotent and cannot be repeated.

POST /api/v1/crm/trial-requests/:id/sessions

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Schedules a free trial class (`TrialSession`) against a trial request — typically the step right after initial contact, before conversion. Also flips the parent trial request's status to `TRIAL_SCHEDULED`. `courseId` is copied automatically from the trial request; `meetingProvider` is always recorded as `MANUAL` (no auto-generated meeting link at this stage).

Path params: - `id` (string, uuid) — the trial request to schedule a session for

Request body: | Field | Type | Required | Notes | |---|---|---|---| | `teacherId` | string (uuid) | no | Must reference an `ACTIVE` teacher profile if provided. | | `scheduledStartAt` | string (ISO date) | yes | Must be strictly before `scheduledEndAt`. | | `scheduledEndAt` | string (ISO date) | yes | | | `timezone` | string | yes | Valid IANA timezone. |

Response (201):

```
{
  "id": "session-uuid",
  "trialRequestId": "tr-uuid",
  "teacherId": "teacher-profile-uuid",
  "courseId": "course-uuid",
  "scheduledStartAt": "2026-09-11T13:00:00.000Z",
  "scheduledEndAt": "2026-09-11T13:30:00.000Z",
  "timezone": "Asia/Karachi",
  "status": "SCHEDULED",
  "notes": null,
  "createdAt": "2026-09-04T10:05:00.000Z",
  "updatedAt": "2026-09-04T10:05:00.000Z"
}
```

Errors: - **404 Not Found** — no trial request with that id. - **400 Bad Request** — **scheduledStartAt** is not strictly before **scheduledEndAt**. - **400 Bad Request** — **teacherId** was provided but does not reference an existing, **ACTIVE** teacher.

GET /api/v1/crm/trial-requests/:id/sessions

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Lists every trial session scheduled for a given trial request, most recently scheduled first (there can be more than one, e.g. after a reschedule).

Path params: - **id** (string, uuid) — the trial request's id

Response (200): array of trial sessions, same shape as the schedule response above. Empty array if none exist (no 404, even for an unknown trial request id).

Errors: none specific.

Trial Sessions

PATCH /api/v1/crm/trial-sessions/:id

Auth: Bearer token required (ADMIN, SUPER_ADMIN — the entire **crm/trial-sessions** controller is class-level role-gated)

Description: Updates a trial session's outcome — typically marking it **COMPLETED**, **CANCELLED**, **NO_SHOW**, or **RESCHEDULED**, with optional notes. When the new status is **COMPLETED**, the parent trial request's status is automatically bumped to **TRIAL_COMPLETED** in the same transaction, ready for the admin to **convert** it.

Path params: - **id** (string, uuid) — the trial session's id

Request body: | Field | Type | Required | Notes | |---|---|---|---| | status | string | no | One of **SCHEDULED**, **COMPLETED**, **CANCELLED**, **NO_SHOW**, **RESCHEDULED**. | | notes | string | no | |

Response (200): the updated trial session, same shape as the schedule-session response.

Errors: - **404 Not Found** — no trial session with that id.

Contact Inquiries

POST /api/v1/crm/contact-inquiries

Auth: Public

Description: The public "Contact Us" form (SRS Section 18) — a lighter-weight, non-course-specific inquiry than a trial request. Created in **NEW** status.

Request body: | Field | Type | Required | Notes | |---|---|---|---| | name | string | yes | Non-empty. | | email | string | yes | Valid email format. | | phone | string | no | | | whatsapp | string | no | | | subject | string | yes | Non-empty. | | message | string | yes | Non-empty. |

Response (201):

```
{
  "id": "ci-uuid",
  "name": "Fatima",
  "email": "fatima@example.com",
  "phone": null,
  "whatsapp": null,
  "subject": "Question about pricing",
  "message": "Do you offer sibling discounts?",
  "status": "NEW",
  "assignedAdminId": null,
  "createdAt": "2026-09-04T10:00:00.000Z",
  "updatedAt": "2026-09-04T10:00:00.000Z"
}
```

Errors: - **400 Bad Request** — **email** is not a valid email address, or a required field is missing.

GET /api/v1/crm/contact-inquiries

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Lists contact inquiries, newest first.

Query params: - **page** (number, optional, default 1) - **limit** (number, optional, default 20, max 100)

Response (200): paginated list, same item shape as the create response.

Errors: none specific beyond generic auth/role failures.

GET /api/v1/crm/contact-inquiries/:id

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Fetches a single contact inquiry by id.

Path params: - **id** (string, uuid)

Response (200): same shape as the create response.

Errors: - **404 Not Found** — no contact inquiry with that id.

PATCH /api/v1/crm/contact-inquiries/:id

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Updates a contact inquiry's triage status and/or assigns it to a staff member.

Path params: - **id** (string, uuid)

Request body: | Field | Type | Required | Notes | |---|---|---|---| | status | string | no | One of **NEW**, **IN_PROGRESS**, **RESOLVED**, **CLOSED**. | | assignedAdminId | string (uuid) | no | |

Response (200): same shape as the create response, with updated fields.

Errors: - **404 Not Found** — no contact inquiry with that id. - **400 Bad Request** — **status** is not one of the allowed values.

Communication

The Communication domain covers three related but independent concerns: direct/group **messaging** between users, admin-authored **announcements** that get pushed to targeted audiences, and a per-user **notification** feed that both of the above (and other domains, e.g. Scheduling) write into. Sending a message or publishing an announcement doesn't just persist a row — it synchronously fans out one **Notification** per affected recipient in the same request, so the notification feed is the one place a user can see "something happened" across every domain without polling each one individually. Right now only the **IN_APP** channel is actually delivered (rows are created with **status: SENT**); **PUSH/EMAIL/SMS/WHATSAPP** rows would be recorded as **PENDING** but nothing dispatches them yet — there's no provider wired up.

POST /api/v1/messaging/conversations

Auth: Bearer token required (any authenticated user)

Description: Starts a new conversation. The caller is always added as a participant even if they omit their own id from **participantUserIds**. If no **type** is given, it's inferred from the resulting participant count: more than 2 participants becomes **GROUP**, otherwise **DIRECT**.

Request body: | Field | Type | Required | Notes | |---|---|---|---| | participantUserIds | string[] (uuid v4) | yes | At least 1 id. The caller's own user id is added automatically if not included. | | type | string | no | One of **DIRECT**, **GROUP**, **INSTITUTE**. Inferred from participant count if omitted. | | title | string | no | Conversation title (mainly useful for **GROUP/INSTITUTE**). |

Response (201): the created conversation.

```
{
  "id": "conv-uuid",
  "type": "DIRECT",
  "title": null,
```

```
"createdAt": "2026-09-04T10:00:00.000Z",
"updatedAt": "2026-09-04T10:00:00.000Z",
"participantUserIds": ["teacher-user-uuid", "student-user-uuid"]
}
```

Errors: - **404 Not Found** — one or more ids in **participantUserIds** don't reference an existing **User**.

GET /api/v1/messaging/conversations/me

Auth: Bearer token required (any authenticated user)

Description: Lists every conversation the caller currently participates in (excludes conversations they've left), newest-activity first.

Response (200): array of conversations, same shape as the create response. Empty array if the caller has none.

Errors: none specific.

GET /api/v1/messaging/conversations/:id/messages

Auth: Bearer token required (must be a current participant of the conversation)

Description: Paginated message history for one conversation, newest first. Soft-deleted messages (**deletedAt** set) are excluded.

Path params: - **id** (string, uuid) — the conversation id

Query params: - **page** (number, optional, default 1) - **limit** (number, optional, default 20, max 100)

Response (200): paginated list.

```
{
  "data": [
    {
      "id": "msg-uuid",
      "conversationId": "conv-uuid",
      "senderId": "teacher-user-uuid",
      "messageType": "TEXT",
      "body": "Assalamu alaikum, your class starts at 5pm today.",
      "createdAt": "2026-09-04T10:01:00.000Z",
      "editedAt": null
    }
  ],
  "meta": { "page": 1, "limit": 20, "total": 1, "totalPages": 1 }
}
```

Errors: - **404 Not Found** — no conversation with that id exists. - **403 Forbidden** — the caller is authenticated but isn't a current participant of this conversation.

POST /api/v1/messaging/conversations/:id/messages

Auth: Bearer token required (must be a current participant of the conversation)

Description: Sends a message into the conversation and bumps the conversation's `updatedAt` (so it resorts to the top of `GET /messaging/conversations/me`). As a side effect, every other current participant gets a `NEW_MESSAGE` notification whose body is the message text truncated to 140 characters.

Path params: - `id` (string, uuid) — the conversation id

Request body: | Field | Type | Required | Notes | |---|---|---|---| | `body` | string | yes | Non-empty message text. | | `messageType` | string | no | One of `TEXT`, `IMAGE`, `FILE`. Defaults to `TEXT`. |

Response (201): the created message (same shape as the items in the messages list above).

Errors: - `404 Not Found` — no conversation with that id exists. - `403 Forbidden` — the caller isn't a current participant.

PATCH /api/v1/messaging/conversations/:id/read

Auth: Bearer token required (must be a current participant of the conversation)

Description: Marks the conversation as read for the caller by stamping their `ConversationParticipant.lastReadAt`. Does not affect other participants or individual messages.

Path params: - `id` (string, uuid) — the conversation id

Response (204): empty body.

Errors: - `404 Not Found` — no conversation with that id exists. - `403 Forbidden` — the caller isn't a current participant.

POST /api/v1/announcements

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Creates an announcement. It always starts as `DRAFT` — it isn't fanned out as notifications and won't appear in anyone's feed until it's separately published via the update endpoint below.

Request body: | Field | Type | Required | Notes | |---|---|---|---| | `title` | string | yes | Non-empty. | | `body` | string | yes | Non-empty. | | `targets` | array of target objects | no | Each target: { `targetType`, `userId?`, `courseId?` }. See below. | | `targets[].targetType` | string | yes (per target) | One of `ALL_USERS`, `ALL_STUDENTS`, `ALL_PARENTS`, `ALL_TEACHERS`, `SPECIFIC_USER`, `COURSE`. | | `targets[].userId` | string (uuid) | no | Only meaningful when `targetType` is `SPECIFIC_USER`. | | `targets[].courseId` | string (uuid) | no | Only meaningful when `targetType` is `COURSE`. |

Each entry in `targets` is validated as its own nested object (`@ValidateNested({ each: true }) + @Type(() => AnnouncementTargetDto)` on `CreateAnnouncementDto.targets`) — this was a real bug fix: a malformed target (e.g. an unrecognized `targetType`, or a target that isn't an object at all) is now correctly rejected with `400 Bad Request` instead of passing validation and later crashing or being silently dropped during publish.

Response (201): the created announcement, `status` is always `DRAFT` on create.

```
{
  "id": "ann-uuid",
  "title": "Ramadan Schedule Update",
  "body": "Classes will shift by 1 hour during Ramadan.",
  "status": "DRAFT",
```

```
"publishedAt": null,
"createdBy": "admin-user-uuid",
"createdAt": "2026-09-04T10:00:00.000Z",
"updatedAt": "2026-09-04T10:00:00.000Z",
"targets": [{ "id": "target-uuid", "targetType": "ALL_STUDENTS", "userId": null, "courseId": null }]
}
```

Errors: - **400 Bad Request** — a **targets** entry has an unrecognized **targetType** or is otherwise malformed (see note above).

GET /api/v1/announcements/admin

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Back-office listing of every announcement regardless of status, newest first — used to manage drafts and review what's already published.

Query params: - **page** (number, optional, default 1) - **limit** (number, optional, default 20, max 100)

Response (200): paginated list, same item shape as the create response.

Errors: none specific.

GET /api/v1/announcements/me

Auth: Bearer token required (any authenticated user)

Description: The caller's personalized announcement feed: **PUBLISHED** announcements whose targets match **ALL_USERS**, the caller's own role bucket (derived from their roles — **STUDENT**→**ALL_STUDENTS**, **PARENT**→**ALL_PARENTS**, **TEACHER**→**ALL_TEACHERS**), or a **SPECIFIC_USER** target naming the caller directly. **COURSE**-targeted announcements are not yet matched against the caller's enrollments here — a deliberate scope trim, not a bug — so a **COURSE** target currently reaches no one's feed via this endpoint.

Response (200): array of matching announcements, newest-published first. Empty array if none match.

Errors: none specific.

PATCH /api/v1/announcements/:id

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Updates an announcement's title/body, or transitions its **status**. The publish side effect only fires on the DRAFT→PUBLISHED transition: setting **status: PUBLISHED** for the first time stamps **publishedAt** and synchronously fans out one **ANNOUNCEMENT** notification per resolved recipient (expanding **ALL_USERS/ALL_STUDENTS/ALL_PARENTS/ALL_TEACHERS/SPECIFIC_USER** targets into concrete user ids; **COURSE** targets are still not expanded here, matching the feed's scope trim above). Re-saving an already-published announcement does not re-fan-out or move **publishedAt**.

Path params: - **id** (string, uuid) — the announcement id

Request body: | Field | Type | Required | Notes | |---|---|---|---| | title | string | no | | | body | string | no | | | status | string | no | One of **DRAFT**, **PUBLISHED**. |

Response (200): the updated announcement, same shape as the create response.

Errors: - **404 Not Found** — no announcement with that id exists.

GET /api/v1/notifications/me

Auth: Bearer token required (any authenticated user)

Description: The caller's own notification feed (all channels, all statuses), newest first — the single place messages and announcements (and other domains) surface as "something happened".

Query params: - **page** (number, optional, default 1) - **limit** (number, optional, default 20, max 100)

Response (200): paginated list.

```
{
  "data": [
    {
      "id": "notif-uuid",
      "type": "NEW_MESSAGE",
      "title": "New message",
      "body": "Assalamu alaikum, your class starts at 5pm today.",
      "channel": "IN_APP",
      "status": "SENT",
      "readAt": null,
      "sentAt": "2026-09-04T10:01:00.000Z",
      "createdAt": "2026-09-04T10:01:00.000Z"
    }
  ],
  "meta": { "page": 1, "limit": 20, "total": 2, "totalPages": 1 }
}
```

Errors: none specific.

PATCH /api/v1/notifications/:id/read

Auth: Bearer token required (any authenticated user)

Description: Marks a single notification as read by stamping **readAt**. Lookup is scoped to { **id**, **userId** } in one query, so a notification id that exists but belongs to someone else is indistinguishable from one that doesn't exist at all.

Path params: - **id** (string, uuid) — the notification id

Response (200): the updated notification, same shape as the list items above.

Errors: - **404 Not Found** — no notification with that id exists for the caller (either it truly doesn't exist, or it belongs to a different user).

PATCH /api/v1/notifications/read-all

Auth: Bearer token required (any authenticated user)

Description: Bulk-marks every currently-unread notification belonging to the caller as read in one `updateMany`.

Response (200): count of rows updated.

```
{ "updated": 3 }
```

Errors: none specific — calling this with nothing unread simply returns `{ "updated": 0 }`.

Billing

The Billing domain covers pricing **packages** (the public price list), **invoices** issued against a student's enrollment (with itemized line items), and **payments** manually recorded against those invoices. There is no live payment gateway integration yet — payments are entered by an admin after reconciling a bank transfer or similar out-of-band payment, not charged directly through this API (`PaymentTransaction` exists in the schema for a future gateway to plug into without reshaping the `Payment` table). An invoice's `status` moves from `PENDING` to `PAID` automatically the moment its cumulative `COMPLETED` payments reach or exceed its `total` — there's no explicit "mark paid" action for the common case.

POST /api/v1/billing/packages

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Creates a pricing package (e.g. "4 classes/week, billed monthly"). Always starts with `status: ACTIVE`.

Request body: | Field | Type | Required | Notes | |---|---|---|---| | name | string | yes | Non-empty. | | description | string | no | | | classesPerPeriod | number (int) | no | Minimum 1. | | classDurationMin | number (int) | no | Minimum 15. | | billingPeriod | string | yes | One of `WEEKLY`, `MONTHLY`, `QUARTERLY`, `YEARLY`. | | price | number | yes | Minimum 0. | | currency | string | yes | 3-letter currency code (e.g. `USD`). |

Response (201): the created package.

```
{
  "id": "pkg-uuid",
  "name": "Hifz Intensive - 4x/week",
  "description": "4 classes per week, 45 minutes each.",
  "classesPerPeriod": 16,
  "classDurationMin": 45,
  "billingPeriod": "MONTHLY",
  "price": 120,
  "currency": "USD",
  "status": "ACTIVE",
  "createdAt": "2026-09-04T10:00:00.000Z",
  "updatedAt": "2026-09-04T10:00:00.000Z"
}
```

Errors: none specific beyond standard field validation.

GET /api/v1/billing/packages

Auth: Public

Description: The public pricing page's data source — every package with `status: ACTIVE`, cheapest first. Used by the marketing site, no login required.

Response (200): array of packages, same shape as the create response.

Errors: none specific.

GET /api/v1/billing/packages/admin

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Back-office listing of every package regardless of status (including `INACTIVE` ones hidden from the public list), newest first.

Query params: - `page` (number, optional, default 1) - `limit` (number, optional, default 20, max 100)

Response (200): paginated list, same item shape as the create response.

Errors: none specific.

PATCH /api/v1/billing/packages/:id

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Updates a package's fields, including toggling `status` between `ACTIVE` and `INACTIVE` (an `INACTIVE` package drops out of the public listing but existing invoices/enrollments referencing it are unaffected).

Path params: - `id` (string, uuid) — the package id

Request body: | Field | Type | Required | Notes | |---|---|---|---| | name | string | no | | | description | string | no | | | classesPerPeriod | number (int) | no | Minimum 1. | | classDurationMin | number (int) | no | Minimum 15. | | billingPeriod | string | no | One of `WEEKLY`, `MONTHLY`, `QUARTERLY`, `YEARLY`. | | price | number | no | Minimum 0. | | status | string | no | One of `ACTIVE`, `INACTIVE`. |

Response (200): the updated package, same shape as the create response.

Errors: - `404 Not Found` — no package with that id exists.

POST /api/v1/billing/invoices

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Issues an invoice against an existing enrollment. `studentId` and `parentId` are not passed in — they're derived server-side from the enrollment's student and that student's primary (`isPrimary: true`) `ParentStudentRelationship`, if one exists. The invoice number is generated server-side in the format `INV-YYYYMMDD-XXXXXX` (today's date, plus 6 random uppercase hex characters) — it is not client-suppliable. Line-item amounts, `subtotal`, and `total` are all computed server-side and rounded to 2 decimal places: each item's `amount = quantity × unitPrice`, `subtotal = Σ item.amount`, `total = subtotal - discount + tax`. The invoice always

starts as **status: PENDING**.

Request body: | Field | Type | Required | Notes | |---|---|---|---| | enrollmentId | string (uuid) | yes | Must reference an existing **Enrollment**. Determines **studentId/parentId** on the invoice. | | packageId | string (uuid) | no | Must reference an existing **Package**, if given. Purely informational on the invoice — doesn't affect pricing. | | currency | string | yes | 3-letter currency code. | | items | array of item objects | yes | At least 1 item. | | items[].description | string | yes (per item) | Non-empty. | | items[].quantity | number (int) | yes (per item) | Minimum 1. | | items[].unitPrice | number | yes (per item) | Minimum 0. | | discount | number | no | Minimum 0. Defaults to 0. Subtracted from **subtotal**. | | tax | number | no | Minimum 0. Defaults to 0. Added after discount. | | dueAt | string (ISO date) | yes | | | notes | string | no | |

Response (201): the created invoice with its items.

```
{
  "id": "inv-uuid",
  "invoiceNumber": "INV-20260904-A1B2C3",
  "studentId": "student-uuid",
  "parentId": "parent-uuid",
  "enrollmentId": "enr-uuid",
  "packageId": "pkg-uuid",
  "currency": "USD",
  "subtotal": 140,
  "discount": 10,
  "tax": 0,
  "total": 130,
  "issuedAt": "2026-09-04T10:00:00.000Z",
  "dueAt": "2026-09-30T00:00:00.000Z",
  "paidAt": null,
  "status": "PENDING",
  "notes": "Created from the Postman collection.",
  "createdAt": "2026-09-04T10:00:00.000Z",
  "updatedAt": "2026-09-04T10:00:00.000Z",
  "items": [
    { "id": "item-uuid", "description": "Hifz Intensive - September", "quantity": 1, "unitPrice": 120, "amount": 120 },
    { "id": "item-uuid-2", "description": "Registration fee", "quantity": 1, "unitPrice": 20, "amount": 20 }
  ]
}
```

Errors: - **404 Not Found** — **enrollmentId** doesn't reference an existing enrollment.

GET /api/v1/billing/invoices

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Back-office listing of every invoice across the platform, newest first.

Query params: - **page** (number, optional, default 1) - **limit** (number, optional, default 20, max 100)

Response (200): paginated list, same item shape as the create response.

Errors: none specific.

GET /api/v1/billing/invoices/me

Auth: Bearer token required (STUDENT only)

Description: The caller's own invoices, resolved from the caller's own `StudentProfile` (not a path parameter, so a student can never pass someone else's id).

Query params: - `page` (number, optional, default 1) - `limit` (number, optional, default 20, max 100)

Response (200): paginated list, same item shape as the create response.

Errors: - `404 Not Found` — the caller's account has no `StudentProfile` yet.

GET /api/v1/billing/invoices/children/:studentId

Auth: Bearer token required (PARENT only)

Description: A specific child's invoices. Access is relationship-checked, not just role-checked: the caller must have a `ParentStudentRelationship` to `studentId` with `canViewPayments: true` (the default when a parent adds a child, but can be turned off per-relationship).

Path params: - `studentId` (string, uuid) — the child's student profile id

Query params: - `page` (number, optional, default 1) - `limit` (number, optional, default 20, max 100)

Response (200): paginated list, same item shape as the create response.

Errors: - `404 Not Found` — the caller's account has no `ParentProfile` yet, or no relationship to `studentId` exists, or one exists but `canViewPayments` is `false` (indistinguishable from "doesn't exist", by design).

GET /api/v1/billing/invoices/:id

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Fetches a single invoice by id, with its items.

Path params: - `id` (string, uuid) — the invoice id

Response (200): single invoice, same shape as the create response.

Errors: - `404 Not Found` — no invoice with that id exists.

PATCH /api/v1/billing/invoices/:id

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Updates an invoice's `status` and/or `notes` directly. This is a manual override path (e.g. cancelling or refunding an invoice) — it does not itself touch payments, and setting `status: PAID` here does not create a payment record.

Path params: - `id` (string, uuid) — the invoice id

Request body: | Field | Type | Required | Notes | |---|---|---|---| | status | string | no | One of `DRAFT`, `PENDING`, `PAID`, `OVERDUE`, `CANCELLED`, `REFUNDED`. | | notes | string | no | |

Response (200): the updated invoice, same shape as the create response.

Errors: - `404 Not Found` — no invoice with that id exists.

POST /api/v1/billing/invoices/:id/payments

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Records a manually-reconciled payment against an invoice (e.g. a bank transfer an admin has confirmed) — this does not charge anything, it just logs that money was received. The new payment always starts **status: COMPLETED**. After recording it, the invoice's **COMPLETED** payments are summed; if that sum is now $\geq$ the invoice's **total**, the invoice is atomically flipped to **status: PAID** and **paidAt** is stamped in the same transaction. There's no explicit overpayment guard — a payment that pushes the cumulative total past the invoice total is still accepted and still triggers **PAID**.

Path params: - **id** (string, uuid) — the invoice id

Request body: | Field | Type | Required | Notes | |---|---|---|---| | amount | number | yes | Minimum 0.01. | | paymentMethod | string | no | Free-text (e.g. **BANK_TRANSFER**). | | notes | string | no | |

Response (201): the created payment.

```
{
  "id": "pay-uuid",
  "invoiceId": "inv-uuid",
  "studentId": "student-uuid",
  "parentId": "parent-uuid",
  "amount": 80,
  "currency": "USD",
  "status": "COMPLETED",
  "paymentMethod": "BANK_TRANSFER",
  "paidAt": "2026-09-04T10:05:00.000Z",
  "notes": "First installment.",
  "createdAt": "2026-09-04T10:05:00.000Z"
}
```

Errors: - **404 Not Found** — no invoice with that id exists. - **400 Bad Request** — the invoice's current **status** is **CANCELLED** or **REFUNDED** ("Cannot record a payment against a cancelled invoice").

GET /api/v1/billing/invoices/:id/payments

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Lists every payment recorded against one invoice, newest first (not paginated — invoices don't accumulate enough payments to need it).

Path params: - **id** (string, uuid) — the invoice id

Response (200): array of payments, same shape as the record-payment response. Empty array if none recorded yet (no 404 for an invoice with zero payments, as long as the invoice itself exists — the endpoint doesn't separately check the invoice exists before querying its payments).

Errors: none specific.

GET /api/v1/billing/payments

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Back-office listing of every payment across every invoice, newest first — for reconciliation/reporting rather than looking at one invoice at a time.

Query params: - `page` (number, optional, default 1) - `limit` (number, optional, default 20, max 100)

Response (200): paginated list, same item shape as the record-payment response.

Errors: none specific.

CMS

The CMS domain manages public-facing website content that staff curate but which needs no login to view: student/parent `Testimonials`, an `Faq` list, and a generic key/value `WebsiteContent` store for arbitrary page copy (hero text, about page, footer, etc. — deliberately not a catch-all for structured data like courses or pricing, which stay in their own domains). All three sub-resources follow the same shape: a `status` field (`DRAFT/PUBLISHED/[HIDDEN]`) gates public visibility, writes are ADMIN/SUPER_ADMIN-only, and reads are split into a public endpoint (published-only) and an admin endpoint (everything, for the CMS editor UI).

Testimonials

POST `/api/v1/cms/testimonials`

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Creates a testimonial. Always starts in `DRAFT` status (`sortOrder` defaults to `0`) — it is not publicly visible until a follow-up `PATCH` sets `status` to `PUBLISHED`.

Request body: | Field | Type | Required | Notes | |---|---|---|---| | `name` | string | yes | Non-empty. | | `countryCode` | string | yes | ISO 3166-1 alpha-2. | | `rating` | number | yes | Integer, 1–5. | | `review` | string | yes | Non-empty. | | `category` | string | no | One of `PARENT`, `ADULT_STUDENT`, `HIFZ_STUDENT`. | | `courseId` | string (uuid) | no | Optionally links the testimonial to a specific course. |

Response (201):

```
{
  "id": "test-uuid",
  "name": "Zainab",
  "countryCode": "GB",
  "rating": 5,
  "review": "My daughter loves her Hifz classes.",
  "category": "PARENT",
  "courseId": "course-uuid",
  "status": "DRAFT",
  "sortOrder": 0,
  "createdAt": "2026-09-04T10:00:00.000Z",
  "updatedAt": "2026-09-04T10:00:00.000Z"
}
```

Errors: - `400 Bad Request` — `rating` outside 1–5, or `category` not one of the allowed values.

GET /api/v1/cms/testimonials

Auth: Public

Description: Lists only **PUBLISHED** testimonials, ordered by **sortOrder** ascending — this is the data behind the public website's testimonials carousel.

Query params: - **page** (number, optional, default 1) - **limit** (number, optional, default 20, max 100)

Response (200): paginated list, same item shape as the create response, filtered to **status: "PUBLISHED"**.

Errors: none specific.

GET /api/v1/cms/testimonials/admin

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Lists every testimonial regardless of status (**DRAFT**, **PUBLISHED**, **HIDDEN**), newest first — the data source for the admin CMS editor's testimonial list. Note this route is registered as a literal path segment **admin**, ahead of nothing else that could collide with it since **GET /:id** does not exist on this controller.

Query params: - **page** (number, optional, default 1) - **limit** (number, optional, default 20, max 100)

Response (200): paginated list, same item shape as the create response, all statuses included.

Errors: none specific beyond generic auth/role failures.

PATCH /api/v1/cms/testimonials/:id

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Updates a testimonial's content and/or moderation state. Setting **status** to **PUBLISHED** is what makes it appear on the public **GET /api/v1/cms/testimonials** endpoint; **HIDDEN** removes it from public view without deleting it.

Path params: - **id** (string, uuid)

Request body: | Field | Type | Required | Notes | |---|---|---|---| | name | string | no | | | rating | number | no | Integer, 1–5. | | review | string | no | | | category | string | no | One of **PARENT**, **ADULT_STUDENT**, **HIFZ_STUDENT**. | | status | string | no | One of **DRAFT**, **PUBLISHED**, **HIDDEN**. | | sortOrder | number | no | Integer, minimum 0. Controls ordering on the public carousel. |

Response (200): the updated testimonial, same shape as the create response.

Errors: - **404 Not Found** — no testimonial with that id. - **400 Bad Request** — **status/category** not one of the allowed values, or **rating** outside 1–5.

DELETE /api/v1/cms/testimonials/:id

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Permanently (hard) deletes a testimonial. There is no soft-delete for this resource — use `status: "HIDDEN"` via `PATCH` instead if the intent is to unpublish without losing the record.

Path params: - `id` (string, uuid)

Response (204): no body.

Errors: - `404 Not Found` — no testimonial with that id.

FAQs

POST /api/v1/cms/faqs

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Creates an FAQ entry. Always starts in `DRAFT` status.

Request body: | Field | Type | Required | Notes | |---|---|---|---| | question | string | yes | Non-empty. | | answer | string | yes | Non-empty. | | category | string | no | Free-text grouping label (e.g. `"Pricing"`, `"Scheduling"`). | | sortOrder | number | no | Integer, minimum 0. Defaults to `0` if omitted. |

Response (201):

```
{
  "id": "faq-uuid",
  "question": "Can I change my class schedule?",
  "answer": "Yes, contact your assigned admin at least 24 hours in advance.",
  "category": "Scheduling",
  "sortOrder": 0,
  "status": "DRAFT",
  "createdAt": "2026-09-04T10:00:00.000Z",
  "updatedAt": "2026-09-04T10:00:00.000Z"
}
```

Errors: - `400 Bad Request` — `question/answer` missing or empty.

GET /api/v1/cms/faqs

Auth: Public

Description: Lists only `PUBLISHED` FAQs, ordered by `sortOrder` ascending — the data behind the public FAQ page. Not paginated (FAQ lists are expected to stay small).

Response (200):

```
[
  {
    "id": "faq-uuid",
    "question": "Can I change my class schedule?",
    "answer": "Yes, contact your assigned admin at least 24 hours in advance.",
    "category": "Scheduling",
    "sortOrder": 0,
    "status": "PUBLISHED",
  }
]
```

```
{
  "createdAt": "2026-09-04T10:00:00.000Z",
  "updatedAt": "2026-09-04T10:10:00.000Z"
}
```

Errors: none specific.

GET /api/v1/cms/faqs/admin

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Lists every FAQ regardless of status, ordered by `sortOrder` ascending — the data source for the admin CMS editor.

Response (200): array, same item shape as the public list, all statuses included.

Errors: none specific beyond generic auth/role failures.

PATCH /api/v1/cms/faqs/:id

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Updates an FAQ's content, ordering, and/or publication status.

Path params: - `id` (string, uuid)

Request body: | Field | Type | Required | Notes | |---|---|---|---| | question | string | no | | | answer | string | no | | | category | string | no | | | sortOrder | number | no | Integer, minimum 0. | | status | string | no | One of `DRAFT`, `PUBLISHED`, `HIDDEN`. |

Response (200): the updated FAQ, same shape as the create response.

Errors: - `404 Not Found` — no FAQ with that id. - `400 Bad Request` — `status` not one of the allowed values.

DELETE /api/v1/cms/faqs/:id

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Permanently (hard) deletes an FAQ. As with testimonials, prefer `status: "HIDDEN"` via `PATCH` to unpublish without losing the record.

Path params: - `id` (string, uuid)

Response (204): no body.

Errors: - `404 Not Found` — no FAQ with that id.

Website Content

GET /api/v1/cms/content/:key

Auth: Public

Description: Fetches a single named content block by its key (e.g. "homepage_hero", "about_page"), but only if it is currently **PUBLISHED** — this is what the public website fetches to render arbitrary copy blocks. A **DRAFT** block with the same key is invisible here even to an authenticated caller (use the admin routes below to preview drafts).

Path params: - **key** (string) — the content block's unique key

Response (200):

```
{
  "id": "wc-uuid",
  "key": "homepage_hero",
  "title": "Learn Quran Online with Certified Teachers",
  "content": "Personalized 1-on-1 Hifz and Tajweed classes...",
  "data": { "ctaLabel": "Book a Free Trial", "ctaHref": "/trial" },
  "status": "PUBLISHED",
  "createdAt": "2026-09-01T09:00:00.000Z",
  "updatedAt": "2026-09-04T10:15:00.000Z"
}
```

Errors: - **404 Not Found** — no content exists under that key, or it exists but is **DRAFT** (not yet published) — the two cases are indistinguishable from this endpoint by design.

GET /api/v1/cms/content/admin/list

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Lists every content block regardless of status, alphabetical by key — the data source for the admin CMS content editor's index page.

Response (200): array, same item shape as the public-by-key response, all statuses included.

Errors: none specific beyond generic auth/role failures.

GET /api/v1/cms/content/admin/:key

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Fetches a single content block by key regardless of status — lets an admin preview a **DRAFT** block before publishing it.

Path params: - **key** (string) — the content block's unique key

Response (200): same shape as the public-by-key response.

Errors: - **404 Not Found** — no content exists under that key.

PUT /api/v1/cms/content/:key

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Creates or updates (upserts) a content block by key. If no block with that key exists yet, one is created; otherwise the existing block's fields are overwritten. This is the only write endpoint for website content — there is no separate **POST**.

Path params: - **key** (string) — the content block's key; created if it doesn't already exist

Request body: | Field | Type | Required | Notes | |---|---|---|---| | title | string | no | | | content | string | no | | Free-text/HTML/markdown body — the caller decides the format. | | data | object | no | Arbitrary JSON for structured extras (e.g. CTA labels, image URLs). | | status | string | yes | One of **DRAFT**, **PUBLISHED**. Unlike testimonials/FAQs, there is no **HIDDEN** state here — only draft vs. live. |

Response (200): the created or updated content block, same shape as the public-by-key response.

Errors: - **400 Bad Request** — **status** missing or not one of **DRAFT/PUBLISHED**.

Platform

The Platform domain covers cross-cutting infrastructure used by every other domain: polymorphic file storage and attachment (**media**), an append-only compliance trail (**audit**), and a small key/value store for runtime configuration (**settings**). None of these model a business entity on their own — they support the People/Courses/Enrollment/etc. domains (e.g. attaching a photo to a teacher profile, or an admin tweaking a site-wide flag).

POST /api/v1/media/upload

Auth: Bearer token required (any authenticated user)

Description: Uploads a file (image or PDF) to storage and creates its **MediaFile** record. Optionally attaches the new file to an entity (e.g. a course) in the same call by including **entityType/entityId**.

Request body: **multipart/form-data**

Field	Type	Required	Notes
file	file	yes	Form field name must be exactly file . Allowed MIME types: image/jpeg , image/png , image/webp , application/pdf . Max size 10MB.

Field	Type	Required	Notes
visibility	string	no	PUBLIC or PRIVATE . Defaults to PRIVATE .
entityType	string	no	One of STUDENT_PROFILE , TEACHER_PROFILE , COURSE , TESTIMONIAL . Provide together with entityId to auto-attach.
entityId	string (uuid)	no	Id of the entity to attach to.

Response (201): the created file's metadata (not the attachment).

```
{
  "id": "b2b9b1e0-...-uuid",
  "originalName": "teacher-photo.jpg",
  "mimeType": "image/jpeg",
  "sizeBytes": 84213,
  "visibility": "PRIVATE",
  "uploadedBy": "3f1c...-uuid",
  "createdAt": "2026-09-03T10:15:00.000Z"
}
```

Errors: - **400 Bad Request** — no file present under the **file** field, or an unsupported MIME type. - **400 Bad Request** — file exceeds the 10MB **fileSize** limit (Multer rejects it before the handler runs).

GET /api/v1/media/entity/:entityType/:entityId

Auth: Bearer token required (any authenticated user)

Description: Lists every media attachment linked to a given entity (e.g. every file attached to a specific course), newest first, each including its parent file's metadata.

Path params: - **entityType** (string) — the entity type the attachments belong to (e.g. **COURSE**) - **entityId** (string, uuid) — the entity's id

Response (200): array of attachments, empty array if none exist (no 404).

```
[
  {
    "id": "attach-uuid",
    "mediaFileId": "b2b9b1e0-...-uuid",
    "entityType": "COURSE",
    "entityId": "course-uuid",
    "createdAt": "2026-09-03T10:20:00.000Z",
    "file": {
      "id": "b2b9b1e0-...-uuid",
      "originalName": "course-thumbnail.png",
```

```

    "mimeType": "image/png",
    "sizeBytes": 40211,
    "visibility": "PUBLIC",
    "uploadedBy": "3f1c...-uuid",
    "createdAt": "2026-09-03T10:15:00.000Z"
  }
}
]

```

Errors: none specific — an unknown/mismatched `entityType/entityId` simply returns an empty array.

GET /api/v1/media/:id

Auth: Bearer token required (any authenticated user, subject to visibility/ownership — see Errors)

Description: Returns a single file's metadata (not its bytes). **PUBLIC** files are readable by anyone authenticated; **PRIVATE** files are only readable by the uploader or an ADMIN/SUPER_ADMIN.

Path params: - `id` (string, uuid) — the media file's id

Response (200): same shape as the upload response.

```

{
  "id": "b2b9b1e0-...-uuid",
  "originalName": "teacher-photo.jpg",
  "mimeType": "image/jpeg",
  "sizeBytes": 84213,
  "visibility": "PRIVATE",
  "uploadedBy": "3f1c...-uuid",
  "createdAt": "2026-09-03T10:15:00.000Z"
}

```

Errors: - **404 Not Found** — no file with that id. - **403 Forbidden** — the file is **PRIVATE**, the caller is not the uploader, and the caller is not ADMIN/SUPER_ADMIN.

GET /api/v1/media/:id/file

Auth: Bearer token required (any authenticated user, same visibility/ownership rule as above)

Description: Streams the raw file bytes with the original **Content-Type** and an **inline Content-Disposition** header carrying the original filename — suitable for direct display (e.g. `<img src>`).

Path params: - `id` (string, uuid) — the media file's id

Response (200): raw binary body; no JSON envelope. Notable response headers: **Content-Type:** `<original mime type>`, **Content-Disposition:** `inline; filename=<originalName>`.

Errors: same as `GET /api/v1/media/:id` — **404 Not Found** if missing, **403 Forbidden** if the visibility/ownership check fails.

DELETE /api/v1/media/:id

Auth: Bearer token required (any authenticated user, subject to ownership — see Errors)

Description: Permanently deletes a file from storage and its database record. Only the uploader or an ADMIN/SUPER_ADMIN may delete a file (visibility does not affect delete permission — only ownership/role does).

Path params: - `id` (string, uuid) — the media file's id

Response (204): no content.

Errors: - `404 Not Found` — no file with that id. - `403 Forbidden` — caller is neither the uploader nor ADMIN/SUPER_ADMIN.

POST /api/v1/media/:id/attach

Auth: Bearer token required (any authenticated user, subject to ownership — see Errors)

Description: Links an already-uploaded file to an entity (e.g. attaching a teacher's uploaded photo to their `TEACHER_PROFILE`). Only the file's uploader or an ADMIN/SUPER_ADMIN may attach it. A single file can be attached to multiple entities via repeated calls.

Path params: - `id` (string, uuid) — the media file's id to attach

Request body: | Field | Type | Required | Notes | |---|---|---|---| | entityType | string | yes | One of `STUDENT_PROFILE`, `TEACHER_PROFILE`, `COURSE`, `TESTIMONIAL`. | | entityId | string (uuid) | yes | Id of the entity to attach to. |

Response (201):

```
{
  "id": "attach-uuid",
  "mediaFileId": "b2b9b1e0-...-uuid",
  "entityType": "COURSE",
  "entityId": "course-uuid",
  "createdAt": "2026-09-03T10:20:00.000Z",
  "file": { "id": "b2b9b1e0-...-uuid", "originalName": "course-thumbnail.png", "mimeType": "image/png", "si
}
```

Errors: - `400 Bad Request` — `entityType` is not one of the allowed values. - `404 Not Found` — no media file with that id. - `403 Forbidden` — caller is neither the uploader nor ADMIN/SUPER_ADMIN.

GET /api/v1/audit/logs

Auth: Bearer token required (SUPER_ADMIN only — the entire `audit/logs` controller is gated at the class level, no ADMIN carve-out)

Description: Lists the append-only audit trail, newest first. Entries are written automatically for every successful mutation on a role-guarded route (login, profile edits, course changes, etc.) — nothing needs to be hand-instrumented per endpoint.

Query params: - `page` (number, optional, default 1) - `limit` (number, optional, default 20, max 100)

Response (200): paginated list.

```
{
  "data": [
    {
      "id": "log-uuid",
      "actorUserId": "3f1c...-uuid",
      "action": "COURSE_UPDATE",
      "entityType": "Course",
      "entityId": "course-uuid",
      "oldValues": { "status": "DRAFT" },
      "newValues": { "status": "PUBLISHED" },
      "ipAddress": "127.0.0.1",
      "userAgent": "PostmanRuntime/7.36.0",
      "createdAt": "2026-09-03T10:25:00.000Z"
    }
  ],
  "meta": { "page": 1, "limit": 20, "total": 143, "totalPages": 8 }
}
```

Errors: - **403 Forbidden** — caller holds ADMIN but not SUPER_ADMIN (unlike `/settings`, there is no ADMIN read access here at all).

GET /api/v1/settings

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Lists every system setting, alphabetical by key. Settings are an arbitrary JSON value under a string key (feature flags, site-wide copy, thresholds, etc.).

Response (200):

```
[
  {
    "id": "setting-uuid",
    "key": "trial_class_duration_minutes",
    "value": { "minutes": 30 },
    "description": "Length of a free trial class",
    "updatedBy": "3f1c...-uuid",
    "createdAt": "2026-01-10T09:00:00.000Z",
    "updatedAt": "2026-06-02T14:12:00.000Z"
  }
]
```

Errors: none specific beyond generic auth/role failures.

GET /api/v1/settings/:key

Auth: Bearer token required (ADMIN, SUPER_ADMIN)

Description: Fetches a single setting by its key.

Path params: - **key** (string) — the setting's unique key

Response (200): same shape as one item from the list endpoint above.

Errors: - **404 Not Found** — no setting exists with that key.

PUT /api/v1/settings/:key

Auth: Bearer token required (SUPER_ADMIN only — this route re-declares a tighter role than the controller's default ADMIN+SUPER_ADMIN read access)

Description: Creates or updates (upserts) a setting by key. This is the one Platform route where ADMIN and SUPER_ADMIN diverge: an ADMIN can read every setting via the two GET routes above but cannot write any of them here.

Path params: - `key` (string) — the setting's key; created if it doesn't already exist

Request body: | Field | Type | Required | Notes | |---|---|---|---| | value | object | yes | Arbitrary JSON object — the setting's value. | | description | string | no | Human-readable note about what the setting controls. |

Response (200): the upserted setting (same shape as GET).

```
{
  "id": "setting-uuid",
  "key": "trial_class_duration_minutes",
  "value": { "minutes": 45 },
  "description": "Length of a free trial class",
  "updatedAt": "3f1c...-uuid",
  "createdAt": "2026-01-10T09:00:00.000Z",
  "updatedAt": "2026-09-03T10:30:00.000Z"
}
```

Errors: - `403 Forbidden` — caller holds ADMIN but not SUPER_ADMIN.